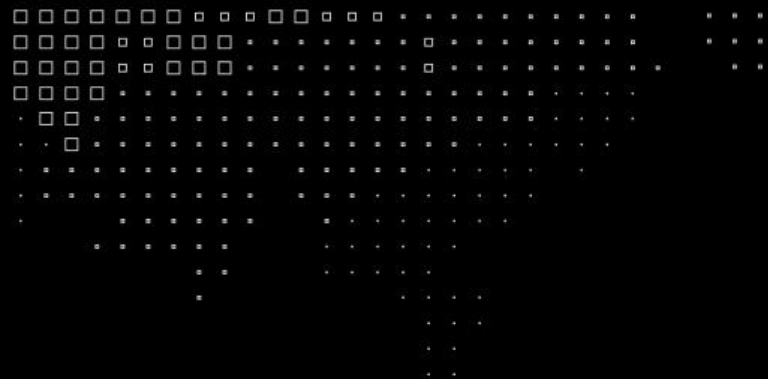


FIAP

NBA

MBA⁺

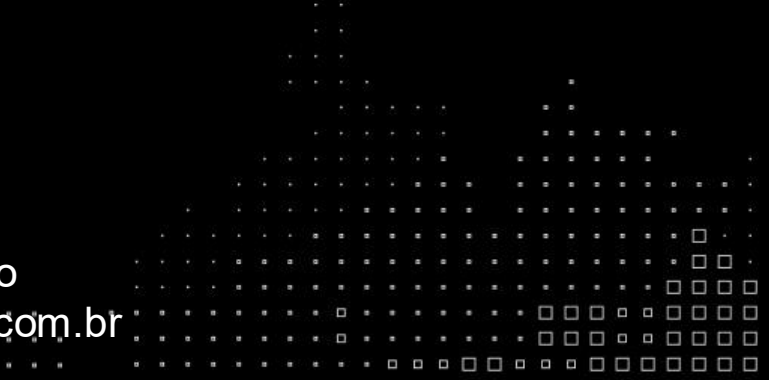
**Engenharia
de Dados**


MBA⁺

MÓDULO 4.
DATA, DEV & GROWTH TECHNICAL SKILLS

Advanced Data Modeling

Prof. Eduardo Galego
profeduardo.galego@fiap.com.br



CALENDÁRIO DE AULAS



Presencial

09/06/2025 (Segunda-feira)

Apresentação da Disciplina

Motivação

Módulo I: Vocabulário de Dados



Presencial

16/06/2025 (Segunda-feira)

Módulo II: Modelagem Relacional – Parte 1
(Diagramas)



Presencial

23/06/2025 (Segunda-feira)

Módulo II: Modelagem Relacional – Parte 2
(Ferramentas)



Presencial

30/06/2025 (Segunda-feira)

Módulo III: Modelagem Dimensional



Remoto

07/07/2025 (Segunda-feira)

Módulo IV: Modelagem Não Relacional

Na aula passada...

1. Diferenças entre os Modelos Relacional e Dimensional;
2. Definição de Business Intelligence;
3. Definição dos Key Performance Indicator;
4. Modelo Dimensional (fato e dimensão);
5. Processo de Modelagem (Conceitual, Lógico e Físico);
6. Tipos de Modelagem: Estrela, Floco de Neve e Constelação;
7. Definição de ETL e Mapeamento de Dados;
8. Arquitetura Física de um projeto de BI;
9. Definição Data Warehouse e Data Marts;
10. Diferença entre DW e Data Lake.

Módulo 4: Modelagem Não Relacional



***Regras para manter
um Banco de Dados
Relacional íntegro:***

A

- Atomicidade

C

- Consistência

I

- Isolamento

D

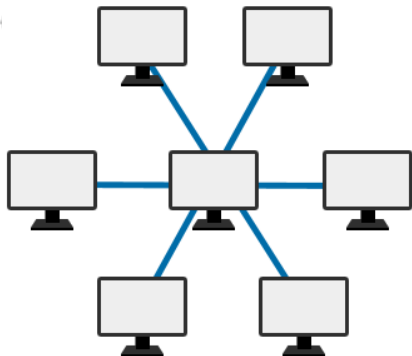
- Duração

No entanto...



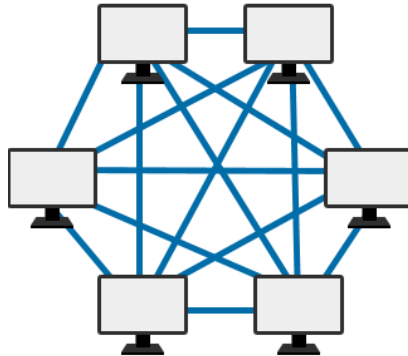
... é possível garantir integridade em aplicação espalhadas pelo globo?

Centralizado



VS

Distribuído



Vantagens:

- Facilidade em acessar e coordenar os dados;
- Possui redundância mínima;
- Mais barato.

Desvantagens:

- Tráfego de dados é maior;
- Em caso de falha, todos os dados estarão inacessíveis ou serão destruídos.

Vantagens:

- Facilmente expandido;
- Acessível de diferentes redes;
- Mais seguro.

Desvantagens:

- Muito caro e difícil de manter devido à sua complexidade;
- Difícil fornecer uma visão uniforme aos utilizadores.

Banco de Dados Centralizado

Banco de Dados Distribuído

Definição

Banco de dados armazenado, localizado e mantido em um único local apenas.

Banco de dados que consiste em vários servidores conectados entre si e espalhados por diferentes locais físicos.

Tempo de acesso

Maior.

Menor.

Gestão de dados

Mais fácil.

Mais difícil.

Visão

Uniforme e completa.

Difícil fornecer uma visão uniforme.

Consistência de dados

Maior consistência de dados.

Este banco de dados pode ter algumas replicações de dados, portanto, a consistência dos dados é menor.

Tolerância à Falhas

Falha implica em total indisponibilidade do sistema.

Falha implica em indisponibilidade parcial do sistema.

Custo

Menos dispendioso.

Muito caro.

Manutenção

Facilidade de manutenção porque a totalidade dos dados e informações estão disponíveis em um único local e, portanto, de fácil acesso.

É difícil manter devido à distribuição de dados e informações em locais variados. Portanto, há a necessidade de verificar problemas de redundância de dados e como manter a consistência dos dados.

Eficiência

Um banco de dados centralizado é menos eficiente à medida que a descoberta de dados se torna bastante complexa devido ao armazenamento de dados e informações em um determinado local.

Um banco de dados distribuído é mais eficiente do que um banco de dados centralizado devido à divisão de dados em vários lugares, o que torna os dados que consomem menos tempo.

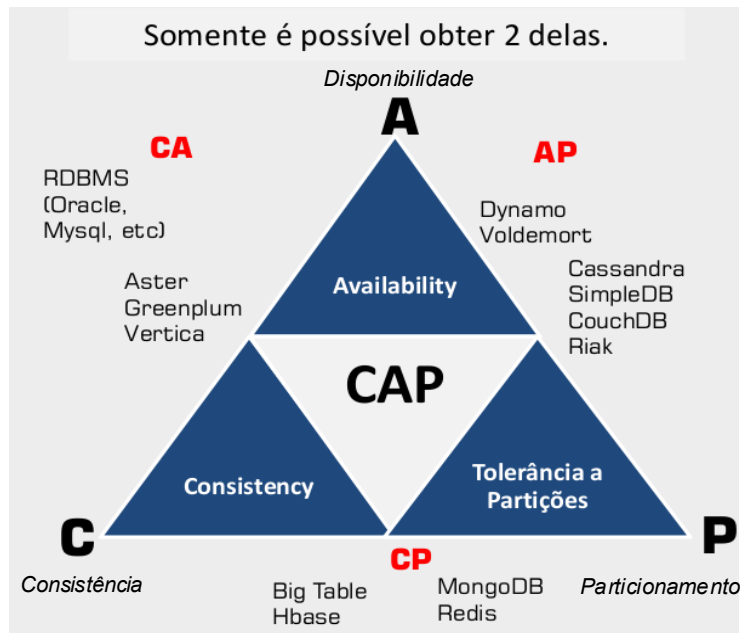
Velocidade de resposta

Maior.

Menor.

Teorema CAP

O **Teorema CAP** diz que é impossível para um sistema de computação distribuída simultaneamente prover todas as três garantias abaixo, **sendo possível somente duas**:

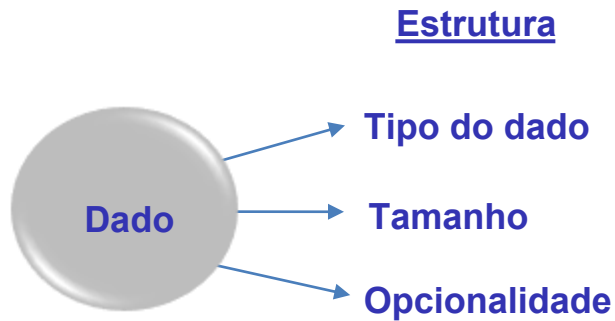


- **Consistência:** todos os nodes vendo os mesmos dados todo o tempo;
- **Disponibilidade:** a garantia de que todo request receba uma resposta, seja bem sucedida, ou falha;
- **Tolerância a particionamento:** o sistema continua a operar apesar de erros de comunicação entre máquinas, ou falhas parciais no sistema.

Relembrando...

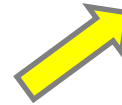


Dados Estruturados



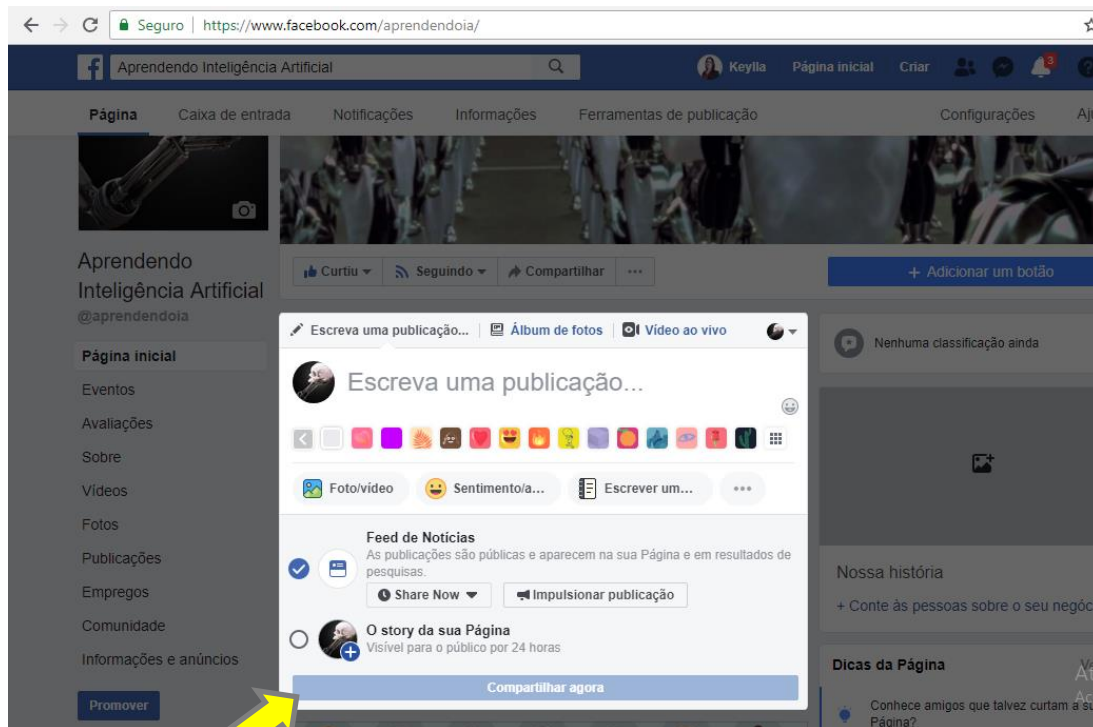
Exemplo

Pessoa	Celular
João	912345678
Maria	
Ana Paula	912345678



Estrutura previamente conhecida

Dados Não Estruturados



Estrutura só será conhecida depois que o dado for gerado

Dados Semi Estruturados

JSON

```
{
  "menu": {
    "id": "file", "value": "File",
    "popup": {
      "menuitem": [
        {
          "value": "New", "onclick": "CreateNewDoc()"
        },
        {
          "value": "Open", "onclick": "OpenDoc()"
        },
        {
          "value": "Close", "onclick": "CloseDoc()"
        }
      ]
    }
  }
}
```

XML

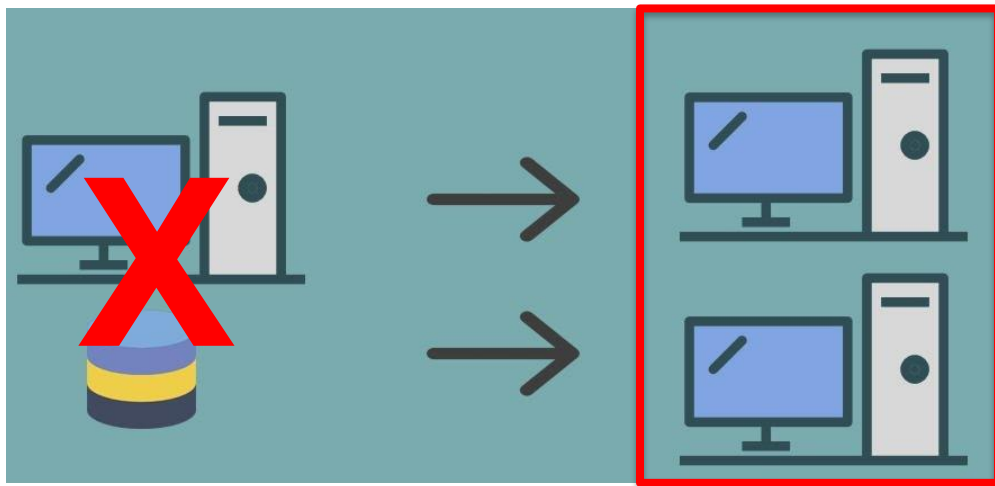
```
<menu id="file" value="File">
  <popup>
    <menuitem value="New" onclick="CreateNewDoc()" />
    <menuitem value="Open" onclick="OpenDoc()" />
    <menuitem value="Close" onclick="CloseDoc()" />
  </popup>
</menu>
```

*Estruturas facilmente
entendível pelo
computador, mas
sem um schema*

Novo Paradigma: NoSQL

Acrônimo de "Not Only SQL".
Traduz-se "Bancos de Dados
Não Relacionais"

Orientado a aplicação em vez de orientado a dados



*Deve-se modelar os dados pensando em como serão
acessados na aplicação.*

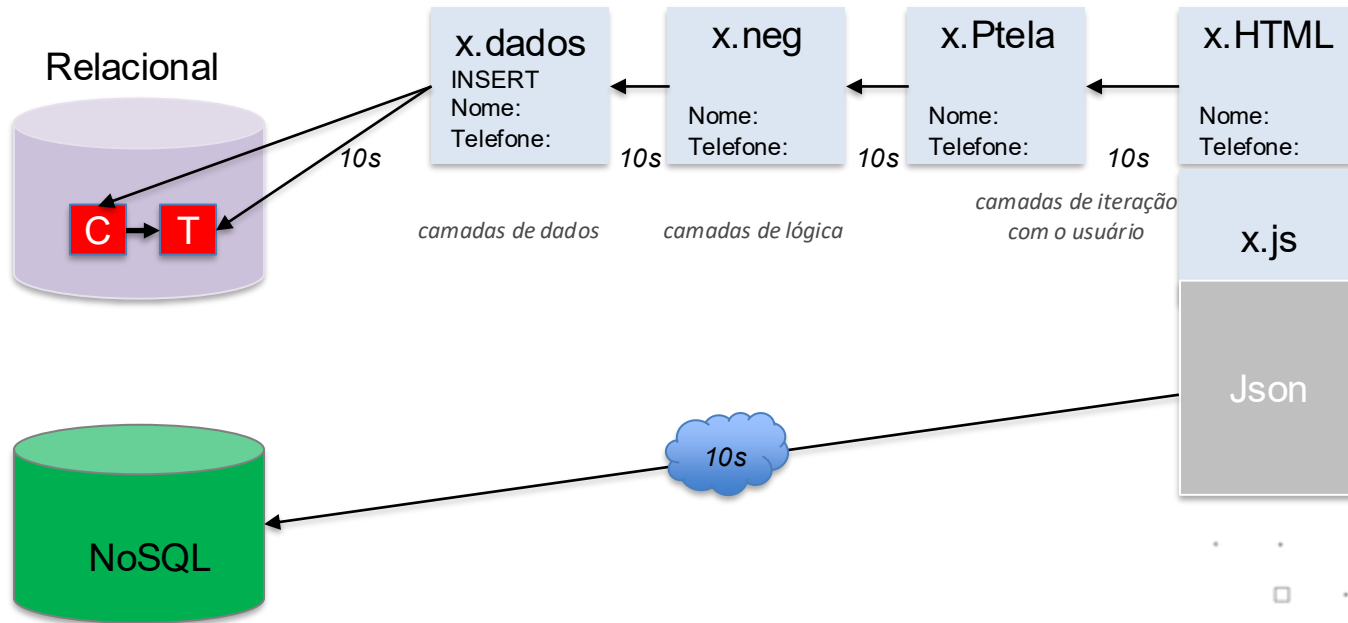
Novo Paradigma: NoSQL

Administrador de dados / DBA

Modelo de dados / Armazenamento

Desenvolvedor

Arquiteturas Multi-Camadas / MVC



Novo Paradigma: NoSQL

1. Chave-valor

Key	Value

- Redis,
- memcached,
- Tokyo
- Cabinet,
- memcachedb,
- Voldemort,
- Simple DB

2. Colunar Multi-Colunar

	C1	C2	C3	C4	C5
A	1			Z	
B			Fr	Z	
C	2		En		
D	3	GE	Z		

- HBase,
- Accumulo,
- Cassandra,
- DynamoDB,
- Google Bigtable,
- SAP Hana

3. Documentos

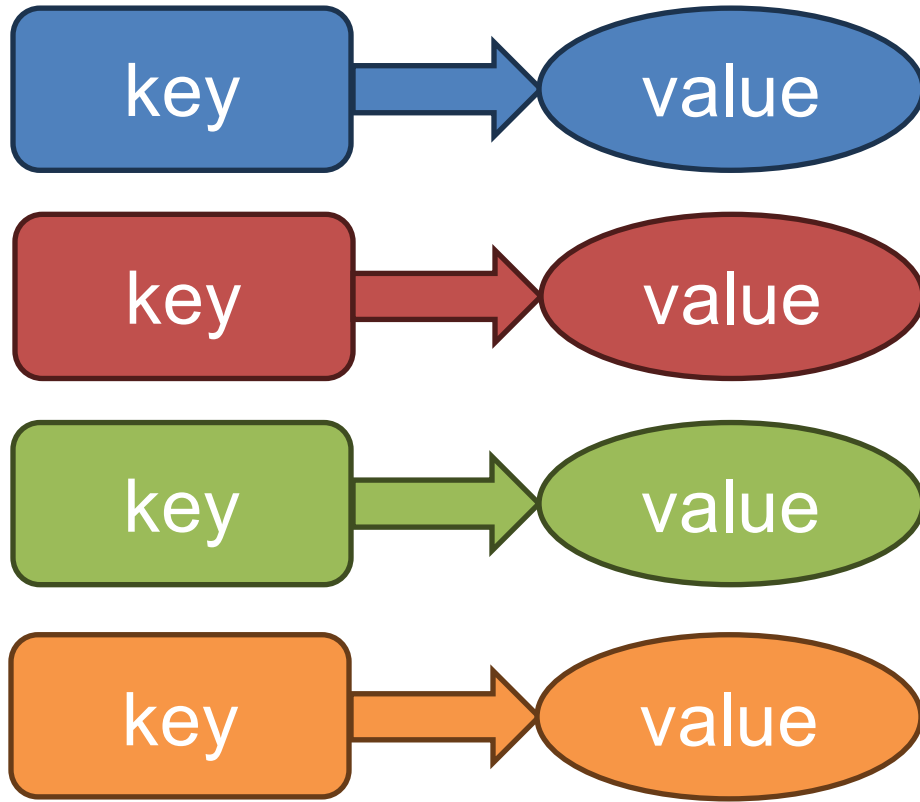


- RavenDB,
- MongoDB
- CouchDB

4. Grafos



- Neo4j,
- Hadoop+Giraph,
- OrientDB,
- InfiniteGraph,
- DEX,
- Teradata Aster

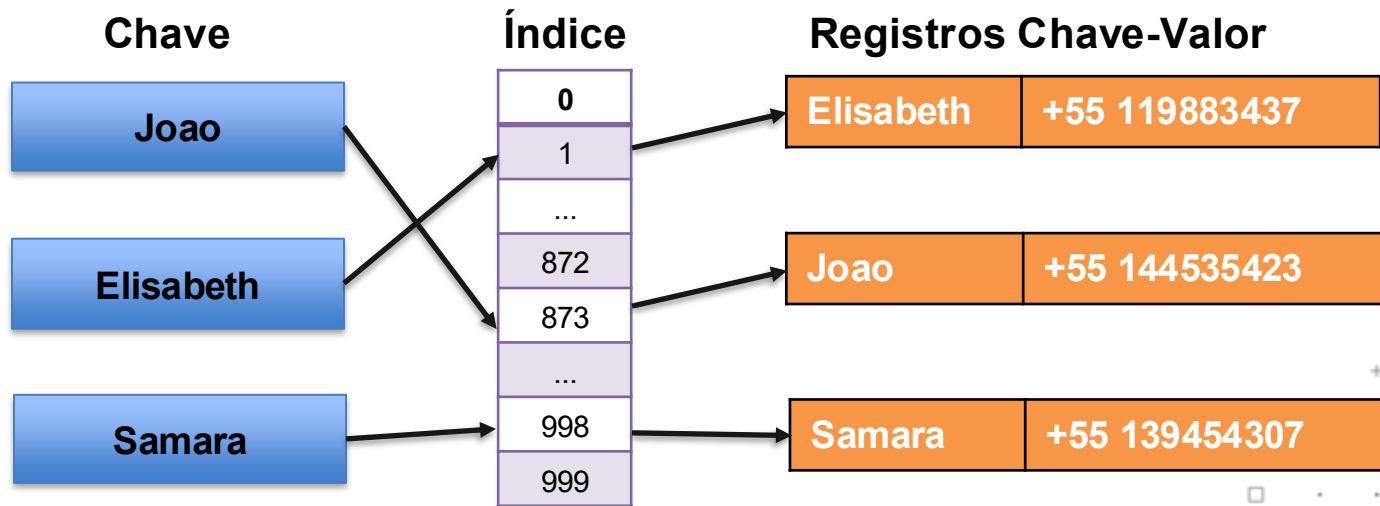


1. NoSQL Chave-valor

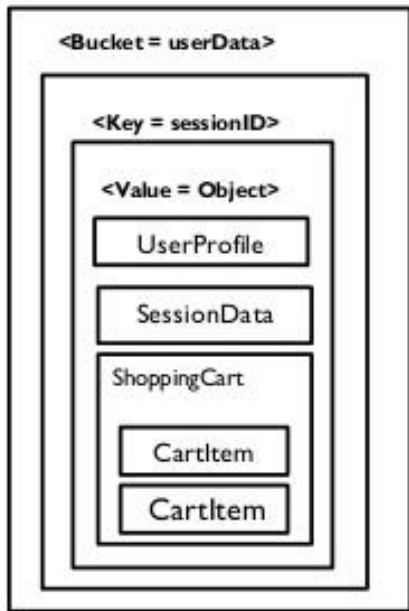


Modelo Chave-valor

- Existe uma coleção de chaves únicas e de valores os quais são associados às chaves
- Um índice único é utilizado para encontrar os valores



Modelo Chave-valor



Fonte: FOWLER e SADALAGE, 2013.

BOM PARA

- Armazenamento de informações de sessão
- Perfis de usuários e preferências
- Dados de carrinhos de compras

RUIM PARA

- Relacionamento entre dados
- Transações com múltiplas operações
- Consultas por dados e atributos
- Operações por conjuntos

Armazenamento Chave-valor

Características

- Armazenamento do dado como pares de chave e valor
- Modelo de dados extremamente simples
- Atributo é chave e conteúdo é valor
- Sem estrutura do modelo:
 - schema-less
- Alta escalabilidade
- Desenhado para manipular carga massiva concorrente

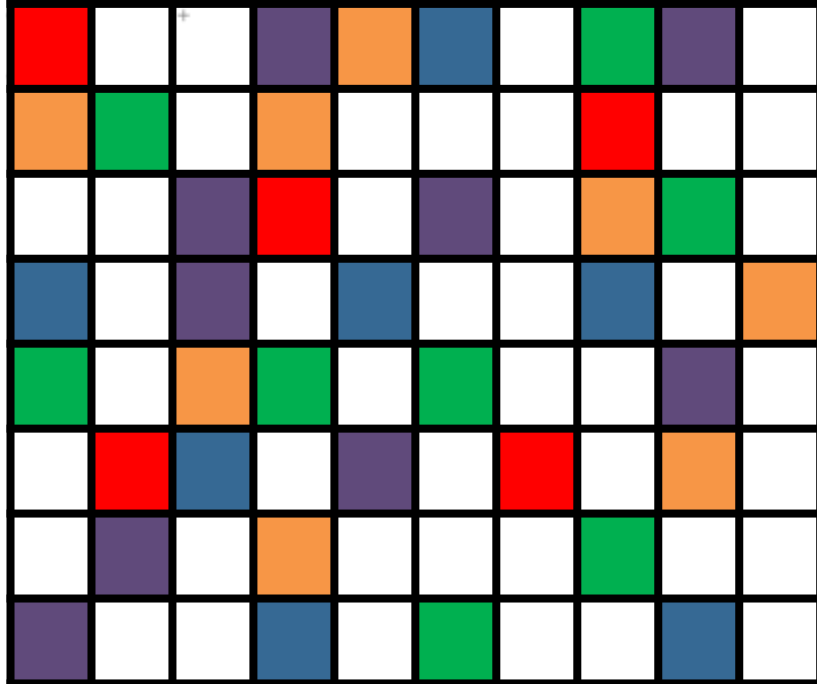
Aplicações / Casos de uso

- Armazenamento de grande quantidade de dados de baixa complexidade
- Gerenciamento e armazenamento de informações de sessão dos usuários
- Armazenamento de informações do carrinho de compra
- Armazenamento de dados financeiros baseados em séries temporais. Necessidade essa muito comum em soluções que atendem o mercado acionário.

Armazenamento Chave-valor



- **Armazenamento de dados em tempo real**
 - As estruturas de dados na memória versáteis do Redis permitem que você crie uma infraestrutura de dados para aplicativos em tempo real que exigem baixa latência e alta taxa de transferência.
- **Cache e armazenamento de sessão**
 - A velocidade do Redis o torna ideal para armazenar em cache consultas de banco de dados, cálculos complexos, chamadas de API e estado de sessão.
- **Transmissão e mensagens**
 - O tipo de dados de fluxo permite ingestão de dados de alta velocidade, mensagens, origem de eventos e notificações.

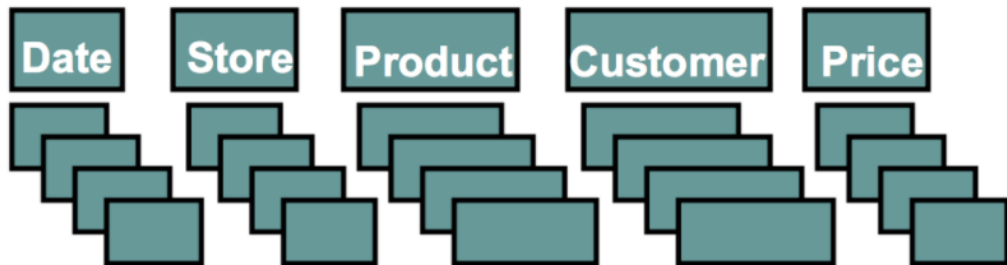


2. NoSQL Colunar

Modelo Colunar (ou Multicolunar)

- **Dados são armazenados em tabelas**

- Você não declara somente colunas, mas **famílias de colunas** e **supercolunas**
- Não há tipos. Tudo é armazenado como binário



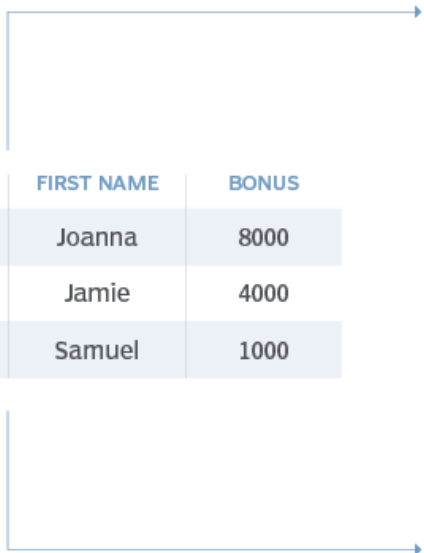
- **Cada linha é uma lista ligada (*Linked List*), ordenada por família de coluna e por nome da coluna**

- Os valores são armazenados dentro das colunas e as **famílias de colunas podem ser diferentes**
- Colunas em branco não ocupam espaço em disco
- Linhas podem ter centenas e até milhares de colunas

- **Não há joins**

- Joins podem ser representados como a chamada de duas famílias de colunas diferentes para uma mesma chave.

Colunas VS Linhas



ID NUMBER	LAST NAME	FIRST NAME	BONUS
513001	Jones	Joanna	8000
502333	Smith	Jamie	4000
455332	Beck	Samuel	1000

Row-oriented database

513001	Jones	Joanna	8000
502333	Smith	Jamie	4000
455332	Beck	Samuel	1000

Column-oriented database

513001	502333	455332
Jones	Smith	Beck
Joanna	Jamie	Samuel
8000	4000	1000

Como Funciona?

Você precisa saber da média de idade por sexo.
E se você tivesse 100 milhões de linhas com 100 colunas ?



Como Funciona?

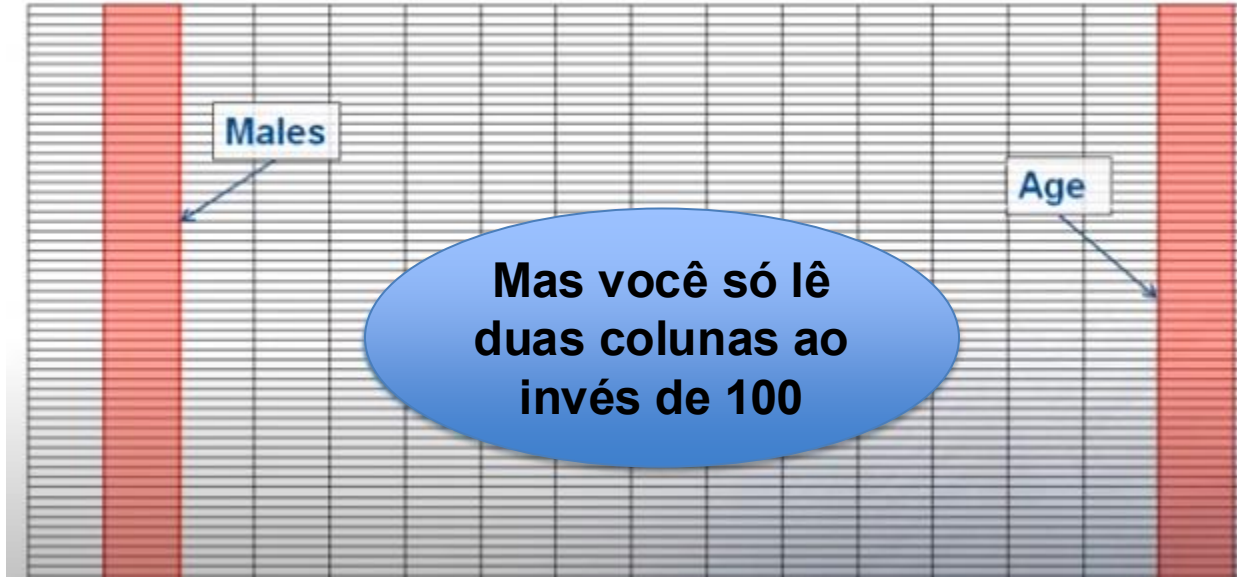
- Cada coluna é armazenada em um arquivo separado

Key	Fname	Lname	State	Zip	Phone	Age	Sex
1	Bugs	Bunny	NY	11217	(718) 938-3235	34	M
2	Yosemite	Sam	CA	95389	(209) 375-6572	52	M
3	Daffy	Duck	NY	10013	(212) 227-1810	35	M
4	Elmer	Fudd	ME	04578	(207) 882-7323	43	M
5	Witch	Hazel	MA	01970	(978) 744-0991	57	F

- Cada coluna de uma determinada linha está no mesmo deslocamento (indexação automática)

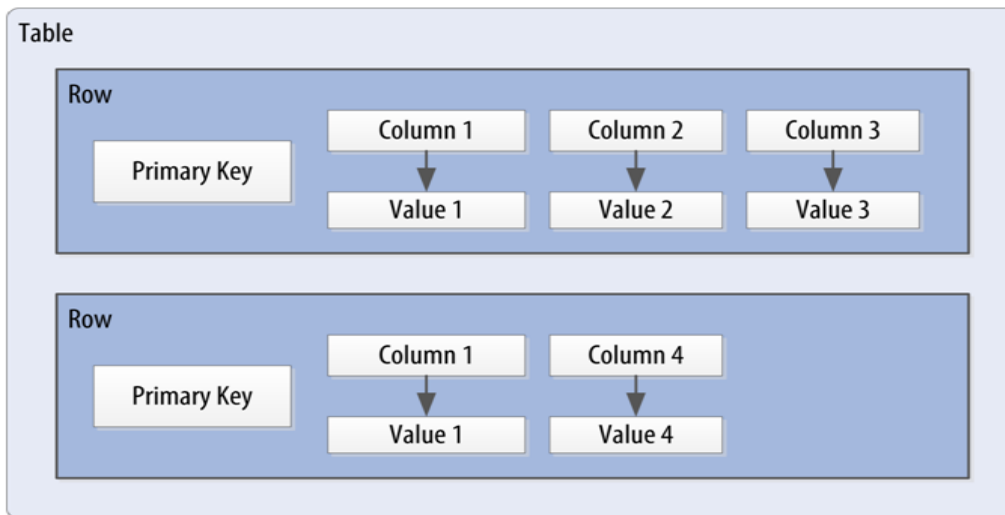
Como Funciona?

Você ainda tem 100 milhões de linhas com 100 colunas ?



Modelagem

Diferente de uma tabela de um banco relacional, no **modelo multicolunar** não é necessário popular todas as colunas ao criar uma nova linha, de forma que podemos ter “larguras” diferentes para cada uma delas (uma coluna de um banco relacional sempre precisa ser preenchida com algum valor, mesmo que seja um valor nulo).



Schemaless

Modelagem

RowId	EmplId	Lastname	Firstname	Salary
001	10	Smith	Joe	60000
002	12	Jones	Mary	80000
003	11	Johnson	Cathy	94000
004	22	Jones	Bob	55000

Orientado a linhas

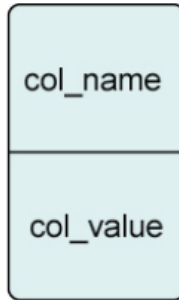
```
001:10,Smith,Joe,60000;
002:12,Jones,Mary,80000;
003:11,Johnson,Cathy,94000;
004:22,Jones,Bob,55000;
```

Orientado a colunas

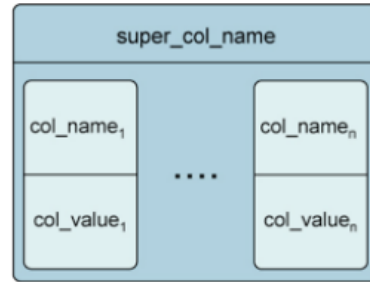
```
10:001,12:002,11:003,22:004;
Smith:001,Jones:002,Johnson:003,Jones:004;
Joe:001,Mary:002,Cathy:003,Bob:004;
60000:001,80000:002,94000:003,55000:004;
```

Modelagem

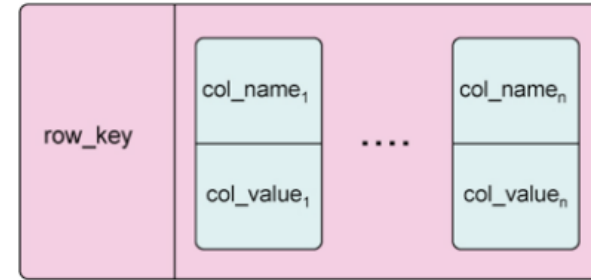
Coluna



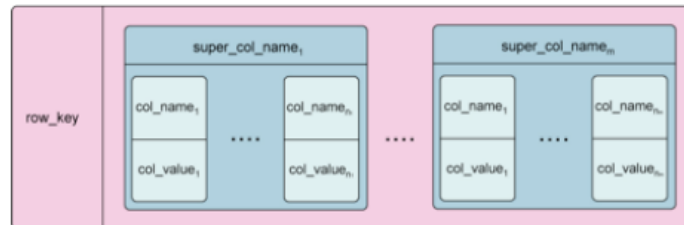
Super Coluna



Família de Colunas



Família de Super Colunas



Modelagem

//coluna

```
{
  Nome: "PrimeiroNome",
  Valor: "Mateus",
  Timestamp: 1234567890
}
```

Chave-Valor

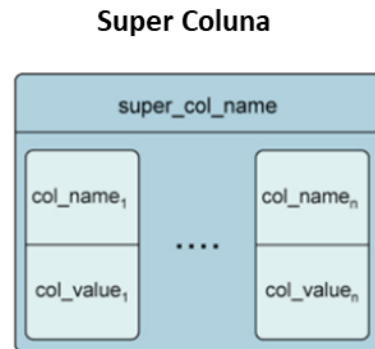
Coluna

col_name

col_value

Modelagem

Consiste em uma coluna cujo valor é um conjunto de colunas



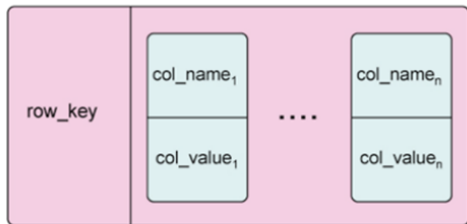
```
{
  Nome: "livro:978-0767905923",
  Valor:
  {
    Autor: "Joel da Silva",
    Titulo: "A Linguagem GeoMDQL",
    Isbn: "978-0767905923"
  }
}
```

Super Col

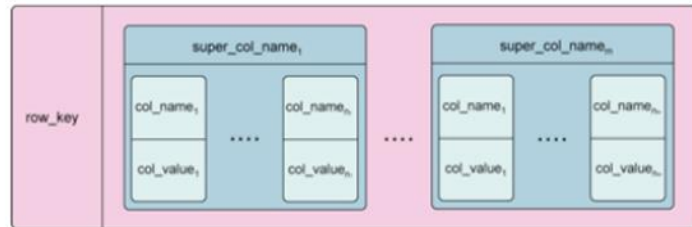
Modelagem

```
// família de colunas
{
  // linha
  "pedro-souza":
  {
    Nome: "Pedro",
    Sobrenome: "Souza",
    UltimaVisita: "12/12/2012"
  },
  // linha
  "marta-alves":
  {
    Nome : "Marta",
    Sobrenome : "Alves",
    Local : "Recife" }
}
```

Família de Colunas



Modelagem



```
// família de supercolunas
```

```
{ // linha
```

```
  Nome: "conta:pedro-souza",
```

```
  Valor: { Endereço: { Nome: "endereço:default",  
                      Valor: { nomeCompleto: "Pedro Souza",  
                               rua: "Av. Boa Viagem 4200",  
                               complemento: "Apto 301" } },
```

```
    Conta: { Nome: "conta:default",
```

```
      Valor: { numeroCartao: "8888-4444-2222-1111",  
              vencimento: "12/2018" } } }
```

```
  } ,
```

```
{ // linha
```

```
  Nome: "conta:marta-alves",
```

```
  Valor: { Endereço: { Nome: "endereço:default",  
                      Valor: { nomeCompleto: "Marta Alves",  
                               rua: "Rua Serinhaem 44",  
                               complemento: "Apto 1801" } } }
```

```
    Conta: { Nome: "conta:default",
```

```
      Valor: { numeroCartao : "9999-7777-5555-1111",  
              vencimento: "12/2020" } } }
```

```
}
```

Super Col

Super Col

Super Col

Super Col

Exemplo

	company					
	name	address		website		
1	DataX	city	San Francisco	protocol	https	
		state	California	domain	datax.com	
		street num	135	subdomain	www	
		street	Kearny St			
2	Process-One	city	Arlington	protocol	https	
		state	Virginia	domain	process1.com	
		street num	3500	subdomain	www	
		street	Wilson St			

super column family

Super Coluna: conjunto de colunas

column

Coluna: Unidade básica de armazenamento que consiste em um par chave-valor

row key

column family

Linha: coleção de colunas associadas a uma chave

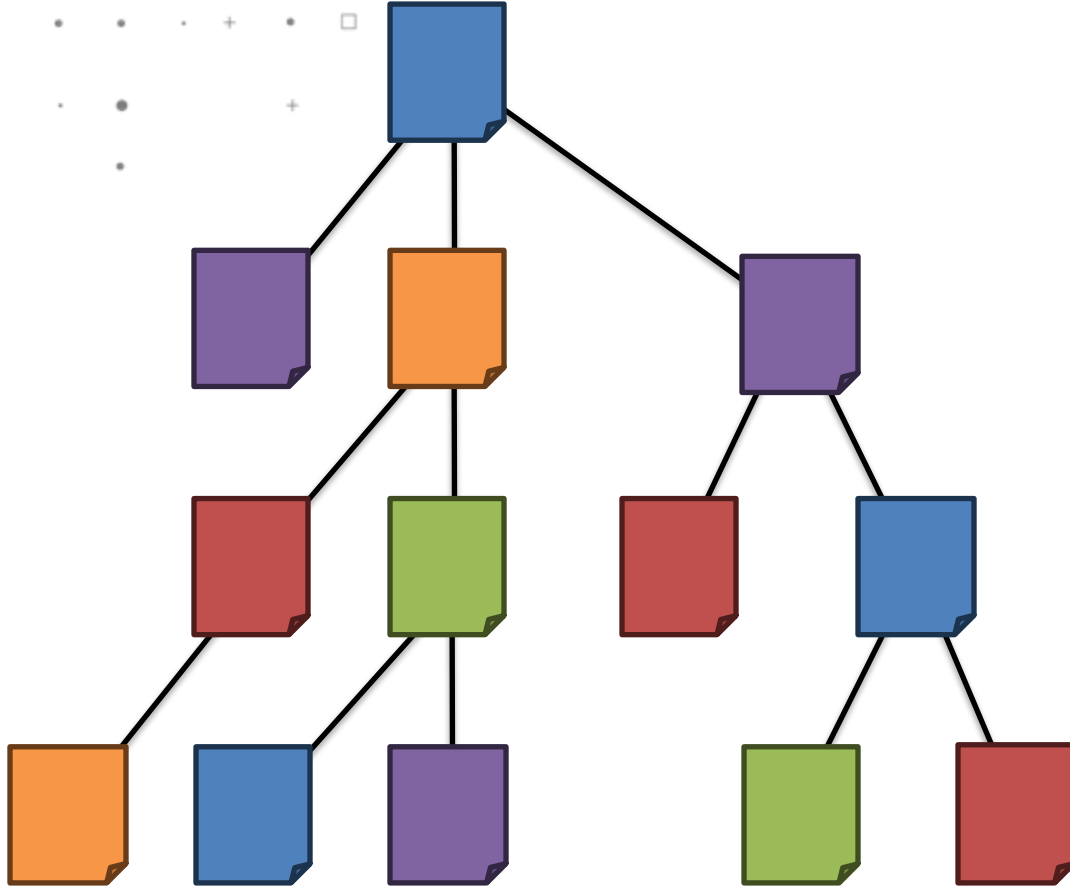
Row Key:
Equivalente a chave primária

Família de colunas:
coleção de linhas semelhantes

Aplicações

Utilizado para operações massivas:

- Como agregações e cálculos
- Aplicações que precisam fazer um número alto de leitura de dados do banco
- Soluções de Data Warehousing com grande volumetria de dados que são altamente consultadas para geração de relatórios e tomada de decisões.



3. NoSQL Documentos

Documentos



Pensando num documento de papel original, vemos **dados de forma semiestruturada**. Pensando num documento como uma ficha de um profissional em uma empresa, lá há vários campos, como nome, telefone, endereço, estado civil, idade, **que são estruturados**. Mas também há **informações não estruturadas**, como a foto do profissional, por exemplo.

Os bancos de documento são **coleções de atributos e valores**, onde um atributo pode ser multi-valorado.

Os bancos de dados orientados a documento não **possuem esquema (schemaless)**, ou seja, os documentos armazenados não precisam possuir estrutura em comum.

Transcrições de Terminologia

SQL Terms/Concepts	MongoDB Terms/Concepts
database	database
table	collection
row	document or BSON document
column	field
index	index
table joins	\$lookup , embedded documents
primary key	primary key

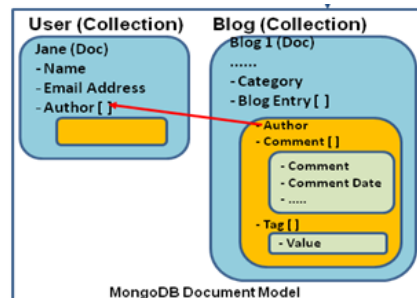
Transcrições entre Modelos

Relacional



```
CREATE TABLE Customer
(First_Name char(50),
Last_Name char(50),
Address char(50),
City char(50),
Country char(25),
Birth_Date datetime);
```

Documento

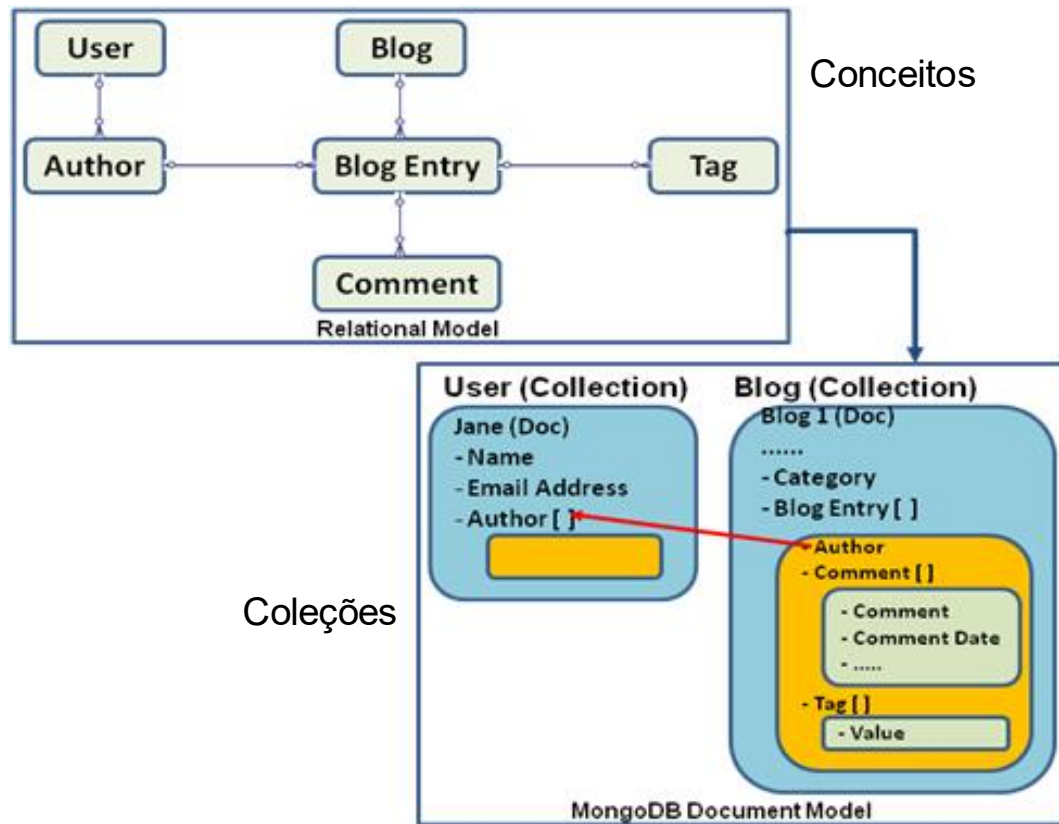


```
{
  "_id": "ObjectID('4bd9e8e17cefd644108961bb)",
  "title": "Adventures in Databases",
  "url": "http://example.com/databases.txt",
  "author": "msmith",
  "vote_count": 20,
  "tags": ["databases", "mongodb", "indexi"],
  "image": {
    "url": "http://example.com/db.jpg",
    "caption": "",
    "type": "jpg",
    "size": 75381,
    "data": "Binary"
  },
  "comments": [
    {
      "user": "bjones",
      "text": "Interesting article!"
    },
    {
      "user": "blogger",
      "text": "Another related article is ."
    }
  ]
}
```

Influência do Modelo Relacional

Técnicas de modelagem de dados RDBMS continuam a desempenhar um papel significativo para aqueles que são novos na tecnologia NoSQL.

O diagrama ao lado ilustra como se pode mapear um modelo de dados relacional para um modelo de dados comparáveis MongoDB:



Exemplo

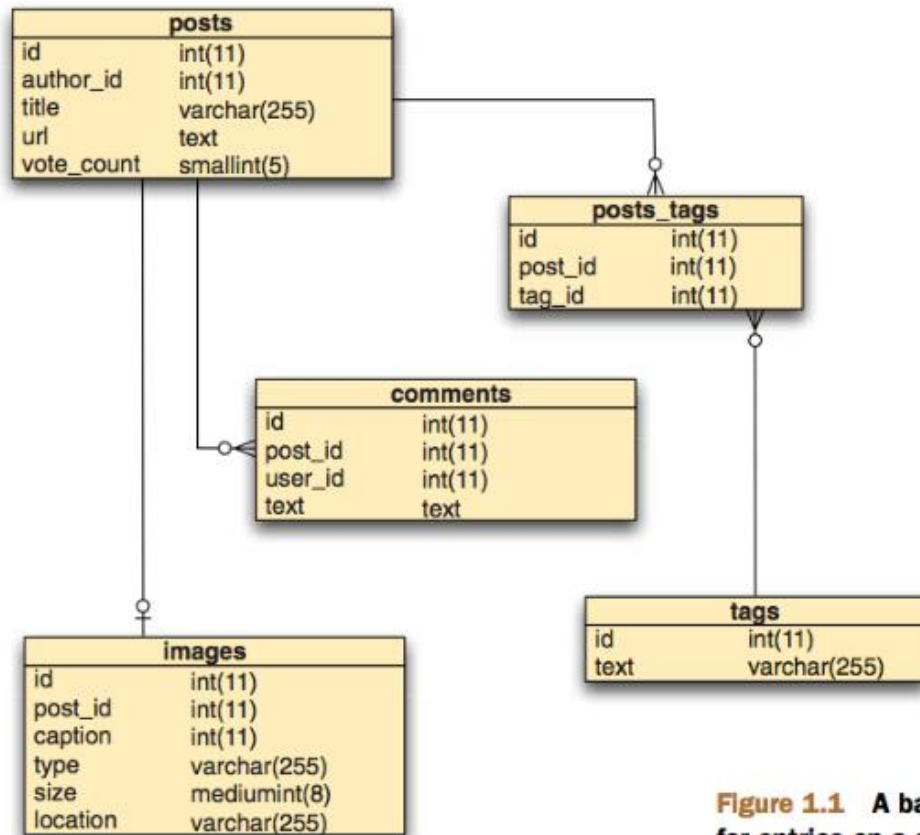


Figure 1.1 A basic relational data model for entries on a social news site

Exemplo

Listing 1.1 A document representing an entry on a social news site

```
{ _id: ObjectID('4bd9e8e17cefd644108961bb'),  
  title: 'Adventures in Databases',  
  url: 'http://example.com/databases.txt',  
  
  author: 'msmith',  
  vote_count: 20,  
  tags: ['databases', 'mongodb', 'indexing'],  
  
  image: {  
    url: 'http://example.com/db.jpg',  
    caption: '',  
    type: 'jpg',  
    size: 75381,  
    data: "Binary"  
  },  
  
  comments: [  
    { user: 'bjones',  
      text: 'Interesting article!'  
    },  
  
    { user: 'blogger',  
      text: 'Another related article is at http://example.com/db/db.txt'  
    }  
  ]  
}
```

_id é uma chave primária

1
Tags são armazenadas
como array de strings

2
Atributos apontando
para outro documento

3
Comentários
armazenado como array

Relacionamento 1:1

```
{
  _id: "joe",
  name: "Joe Bookreader"
}

{
  patron_id: "joe",
  street: "123 Fake Street",
  city: "Faketon",
  state: "MA",
  zip: "12345"
}
```

```
{
  _id: "joe",
  name: "Joe Bookreader",
  address: {
    street: "123 Fake Street",
    city: "Faketon",
    state: "MA",
    zip: "12345"
  }
}
```

Relacionamento 1:N

```
{
  _id: "joe",
  name: "Joe Bookreader"
}

{
  patron_id: "joe",
  street: "123 Fake Street",
  city: "Faketon",
  state: "MA",
  zip: "12345"
}

{
  patron_id: "joe",
  street: "1 Some Other Street",
  city: "Boston",
  state: "MA",
  zip: "12345"
}
```

```
{
  _id: "joe",
  name: "Joe Bookreader",
  addresses:
  [
    {
      street: "123 Fake Street",
      city: "Faketon",
      state: "MA",
      zip: "12345"
    },
    {
      street: "1 Some Other Street",
      city: "Boston",
      state: "MA",
      zip: "12345"
    }
  ]
}
```

Relacionamento 1:N (por referência)

```
{
  title: "MongoDB: The Definitive Guide",
  author: [ "Kristina Chodorow", "Mike Dirolf" ],
  published_date: ISODate("2010-09-24"),
  pages: 216,
  language: "English",
  publisher: {
    name: "O'Reilly Media",
    founded: 1980,
    location: "CA"
  }
}

{
  title: "50 Tips and Tricks for MongoDB Developer",
  author: "Kristina Chodorow",
  published_date: ISODate("2011-05-06"),
  pages: 68,
  language: "English",
  publisher: {
    name: "O'Reilly Media",
    founded: 1980,
    location: "CA"
  }
}
```

```
{
  _id: "oreilly",
  name: "O'Reilly Media",
  founded: 1980,
  location: "CA"
}

{
  _id: 123456789,
  title: "MongoDB: The Definitive Guide",
  author: [ "Kristina Chodorow", "Mike Dirolf" ],
  published_date: ISODate("2010-09-24"),
  pages: 216,
  language: "English",
  publisher_id: "oreilly"
}

{
  _id: 234567890,
  title: "50 Tips and Tricks for MongoDB Developer",
  author: "Kristina Chodorow",
  published_date: ISODate("2011-05-06"),
  pages: 68,
  language: "English",
  publisher_id: "oreilly"
}
```


Atividade Participativa

Transformar as tabelas em um modelo de documentos:

Relacional

Customer

Customer ID	First Name	Last Name	City
0	John	Doe	New York
1	Mark	Smith	San Francisco
2	Jay	Black	Newark
3	Meagan	White	London
4	Edward	Daniels	Boston

Phones

Phone Number	Type	DNC	Customer ID
1-212-555-1212	home	T	0
1-212-555-1213	home	T	0
1-212-555-1214	cell	F	0
1-212-777-1212	home	T	1
1-212-777-1213	cell	(null)	1
1-212-888-1212	home	F	2

Atividade Participativa (Resposta)

Transformar as tabelas em um modelo de documentos:

Relacional

Customer

Customer ID	First Name	Last Name	City
0	John	Doe	New York
1	Mark	Smith	San Francisco
2	Jay	Black	Newark
3	Meagan	White	London
4	Edward	Daniels	Boston

Phones

Phone Number	Type	DNC	Customer ID
1-212-555-1212	home	T	0
1-212-555-1213	home	T	0
1-212-555-1214	cell	F	0
1-212-777-1212	home	T	1
1-212-777-1213	cell	(null)	1
1-212-888-1212	home	F	2



Documento

```
{ customer_id : 1,
  first_name : "Mark",
  last_name : "Smith",
  city : "San Francisco",
  phones: [
    {
      number : "1-212-777-1212",
      dnc : true,
      type : "home"
    },
    {
      number : "1-212-777-1213",
      type : "cell"
    }
  ]
}
```

Atividade Participativa

Transformar as tabelas em um modelo de documentos:

Relacional

customer

ID	first_name	last_name	cell	city	year_of_birth	location_x	location_y
1	'Mary'	'Jones'	'516-555-2048'	'Long Island'	1986	'-73.9876'	'40.7574'

profession

ID	user_id	profession
10	1	'Developer'
11	1	'Engineer'

app

ID	user_id	name	version
20	1	'MyApp'	1.0.4
21	1	'DocFinder'	2.5.7

car

ID	user_id	make	year
30	1	'Bentley'	1973
31	1	'Rolls Royce'	1965

Atividade Participativa (Resposta)

Transformar as tabelas em um modelo de documentos:

Relacional

customer

ID	first_name	last_name	cell	city	year_of_birth	location_x	location_y
1	'Mary'	'Jones'	'516-555-2048'	'Long Island'	1986	'-73.9876'	'40.7574'

profession

ID	user_id	profession
10	1	'Developer'
11	1	'Engineer'

app

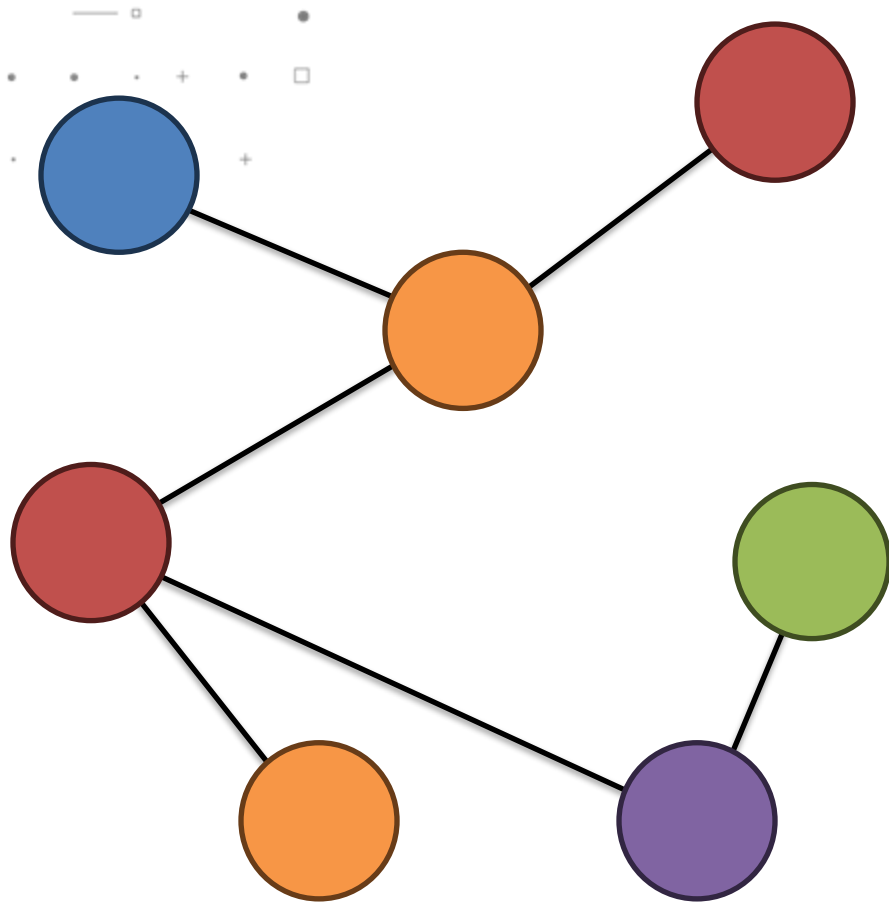
ID	user_id	name	version
20	1	'MyApp'	1.0.4
21	1	'DocFinder'	2.5.7

car

ID	user_id	make	year
30	1	'Bentley'	1973
31	1	'Rolls Royce'	1965

Documento

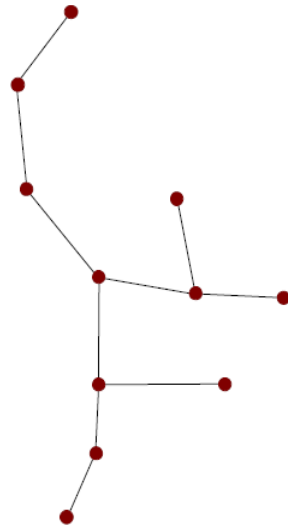
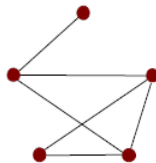
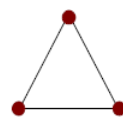
```
{
  first_name: "Mary",
  last_name: "Jones",
  cell: "516-555-2048",
  city: "Long Island",
  year_of_birth: 1986,
  location: {
    type: "Point",
    coordinates: [-73.9876, 40.7574]
  },
  profession: ["Developer", "Engineer"],
  apps: [
    { name: "MyApp",
      version: 1.0.4 },
    { name: "DocFinder",
      version: 2.5.7 }
  ],
  cars: [
    { make: "Bentley",
      year: 1973 },
    { make: "Rolls Royce",
      year: 1965 }
  ]
}
```



4. NoSQL Grafos

Conceito

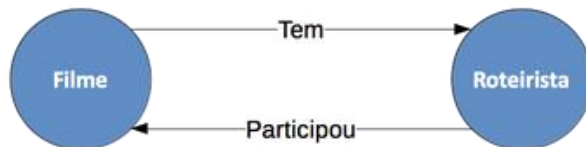
- Objeto abstrato
- Dois tipos de entidades
 - Nós ou *Vértices*
 - Ramos ou *Arestas*
- Vértices representam
 - cidades, pessoas, máquinas, números, etc
- Arestas representam
 - existência de ligações entre nós
 - valor da ligação entre nós
 - distância entre nós



Conceito

Observando a imagem dá pra perceber qual modelo é mais natural?

Modelo em Grafo



Modelo em Relacional



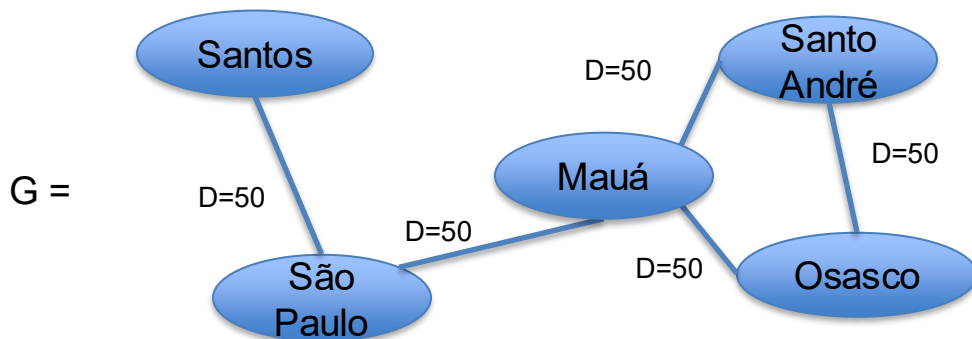
Hoje o mundo real apresenta domínios ricos, complexos, conectados e muitas vezes semi estruturados. O banco de dados de grafos além de representar melhor dados e relacionamentos, possui uma performance superior aos bancos relacionais para **consultas complexas com vários níveis de relações**.

Conceito

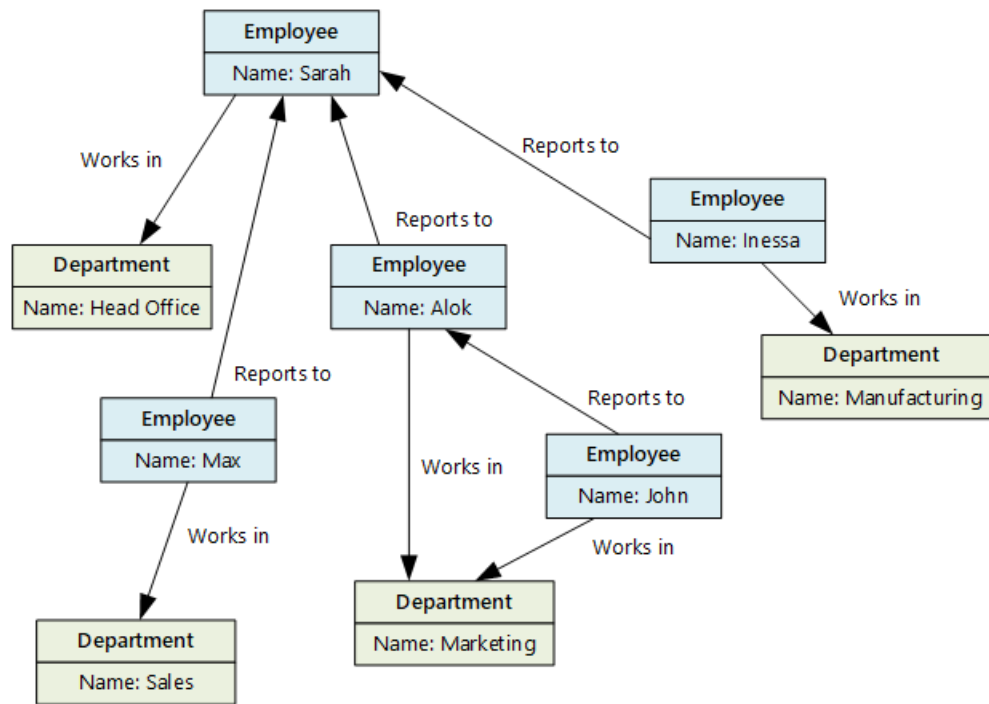
Considere um distrito de cidades $G=(V,E)$, onde as cidades são os vértices e as estradas que as ligam são as arestas

$V = \{\text{Santo André, Santos, Mauá, Osasco, São Paulo}\}$

$E = \{\{\text{Santos, São Paulo}\}, \{\text{São Paulo, Mauá}\}, \{\text{Mauá, Santo André}\}, \{\text{Santo André, Osasco}\}, \{\text{Osasco, Mauá}\}\}$



Modelagem

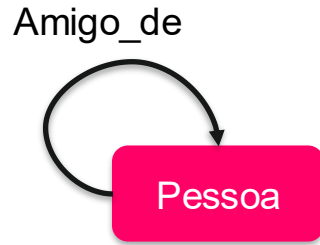


Modelagem

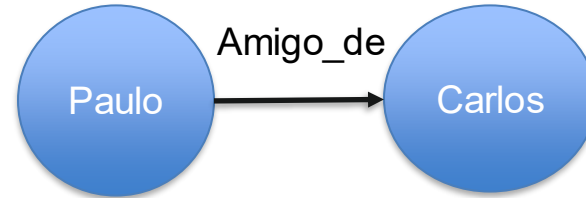


Modelagem

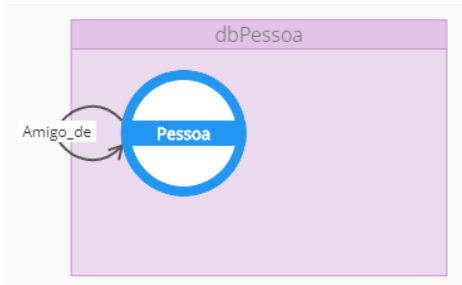
Esquema



Instância



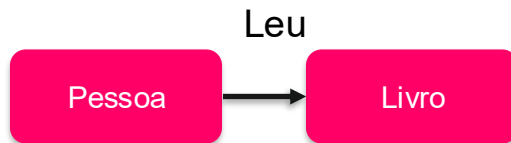
Hackolad



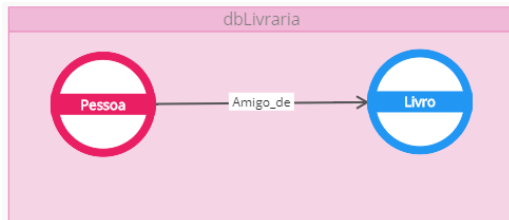
Modelagem

Esquema

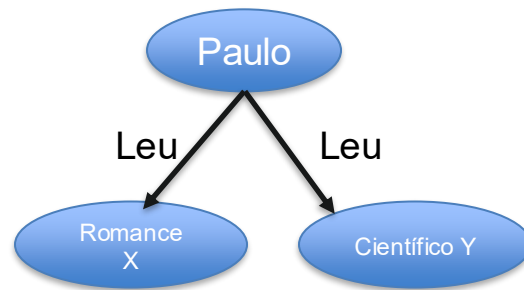
MDG-NoSQL



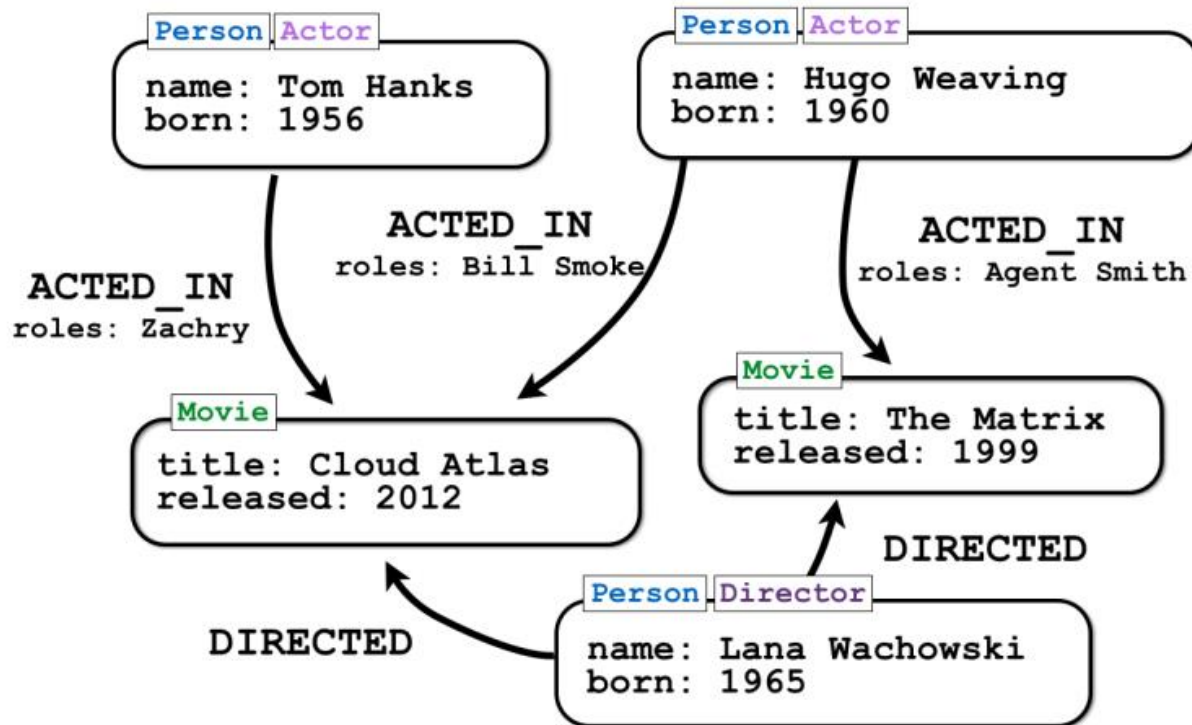
Hackolad



Instância

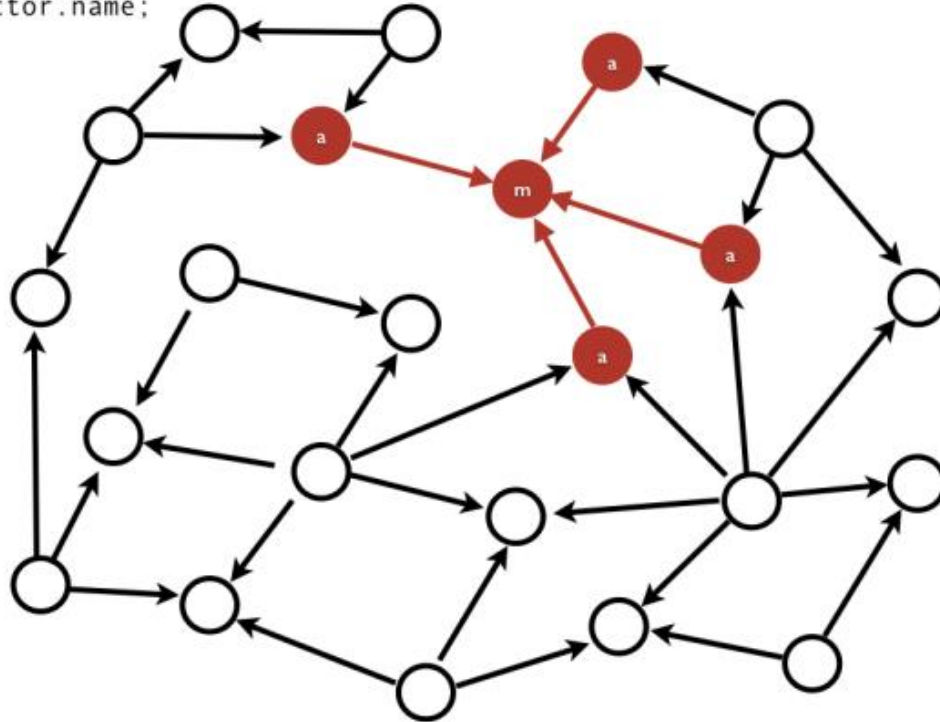


Modelagem (Neo4J)

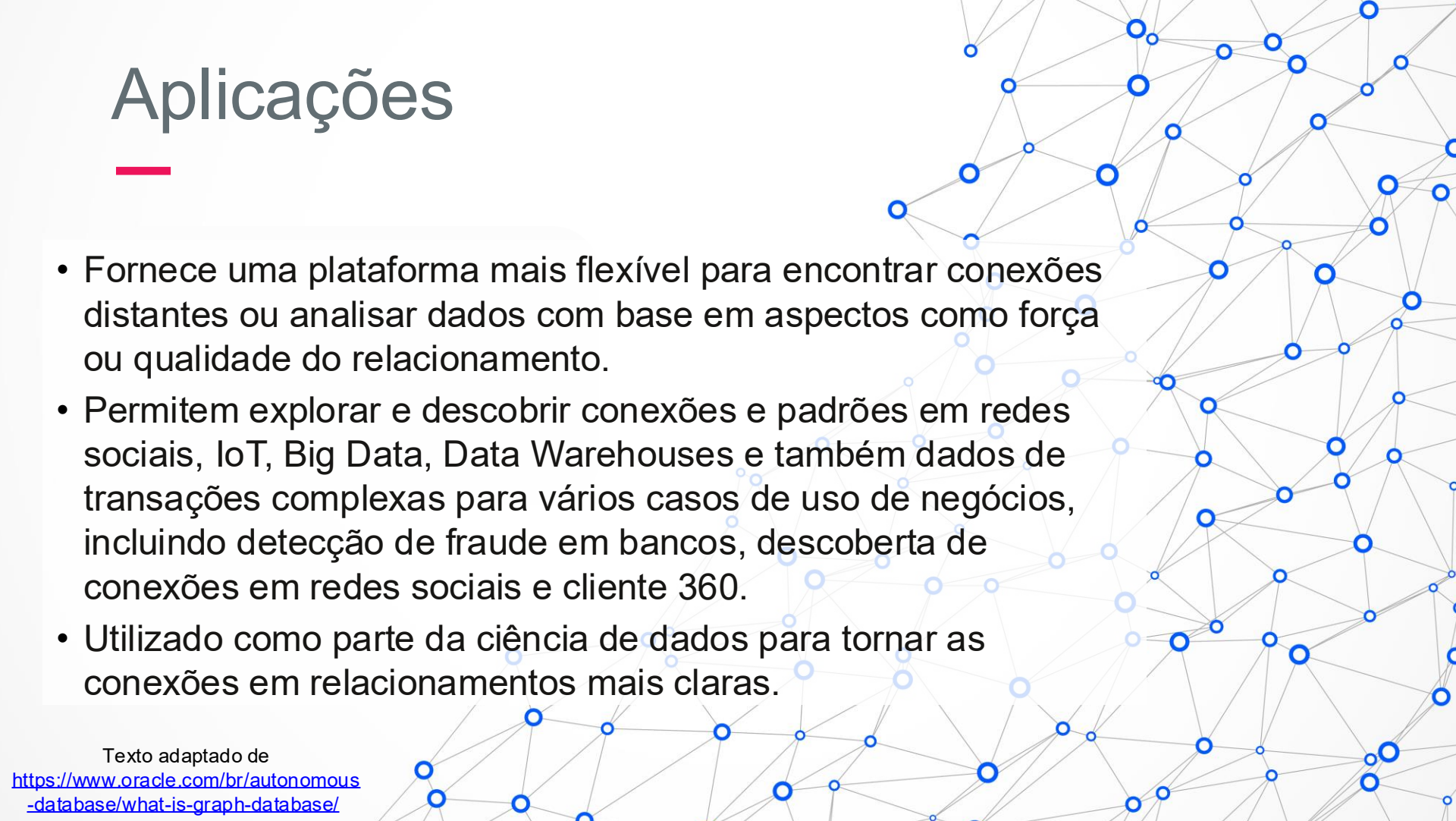


Modelagem (Neo4J)

```
MATCH (movie:Movie)<-[role:ACTED_IN]->(actor:Person)
WHERE movie.title="The Matrix"
RETURN role.roles, actor.name;
```



Aplicações



- Fornece uma plataforma mais flexível para encontrar conexões distantes ou analisar dados com base em aspectos como força ou qualidade do relacionamento.
- Permitem explorar e descobrir conexões e padrões em redes sociais, IoT, Big Data, Data Warehouses e também dados de transações complexas para vários casos de uso de negócios, incluindo detecção de fraude em bancos, descoberta de conexões em redes sociais e cliente 360.
- Utilizado como parte da ciência de dados para tornar as conexões em relacionamentos mais claras.

Texto adaptado de

<https://www.oracle.com/br/autonomous-database/what-is-graph-database/>

Aplicações

PGQL:

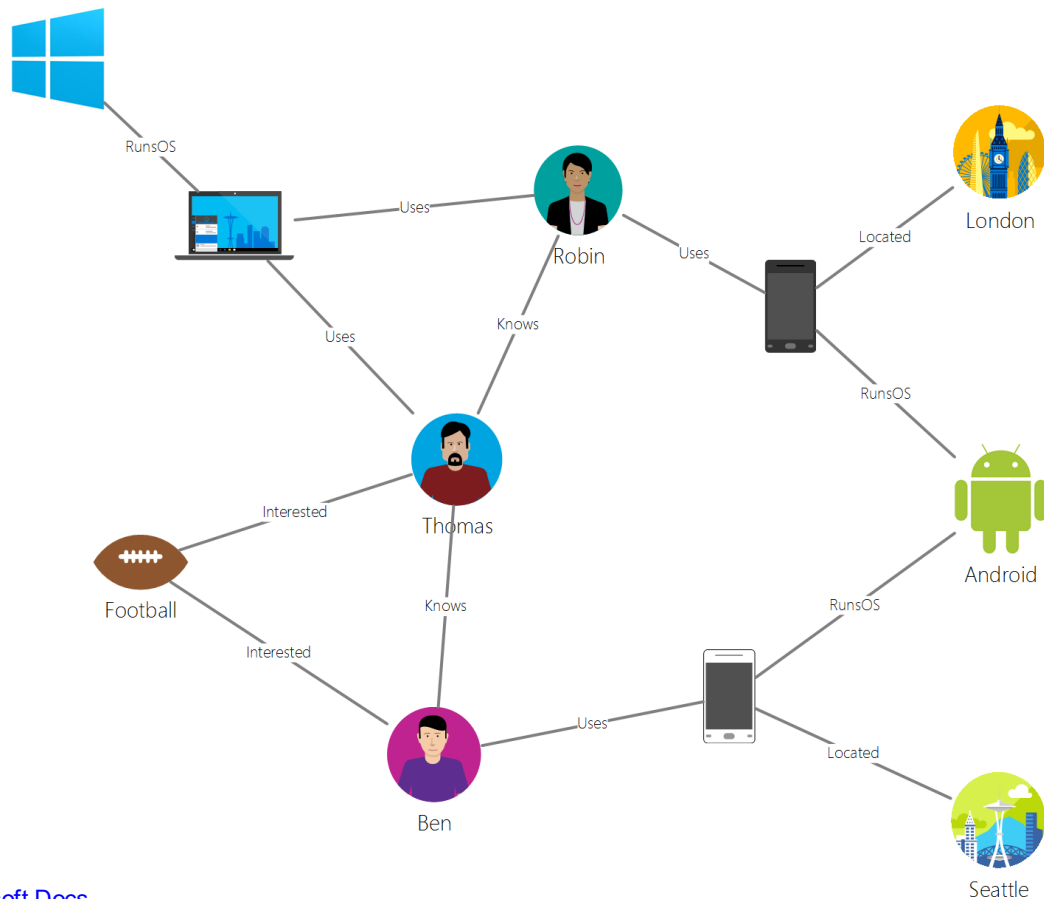
```
PATH shares_movie_with AS (from) <- (acted_in) -> (to)
SELECT y.name
MATCH (x:Actor) -/:shares_movie*/->(y:Actor)
WHERE x.name = 'Iron Man'
AND x <> y
```

SQL Equivalent:

```
WITH temp(actor_id, actor_name) AS (
  —Anchor member:
  SELECT actor_id, name
  FROM Devices
  WHERE name = 'Iron Man'
  UNION ALL
  —Recursive member:
  SELECT Actors.actor_id, Actors.name
  FROM temp, Actors, Connections conn 1,
        Connections conn2, Movies
  WHERE temp.actor_id = conn1.to_actor_id
        AND conn1.from_acted_in_id = Connectors.movies_id
        AND Connectors.movie_id = conn2.from_movie_id
        AND conn2.to_actor_id = Devices.actor_id
        AND temp.actor_id != Actors.actor_id)
CYCLE actor_id SET cycle TO 1 DEFAULT 0
SELECT DISTINCT actor_name
FROM temp
WHERE cycle = 0
AND actor_name <> 'Iron Man'
```


Atividade Participativa

Crie na Hackolade um modelo Grafo para o cadastro dos dados do grafo ao lado:



Com tantas opções de modelagem de dados, qual devo utilizar?



SQL ou NoSQL? Eis a questão!

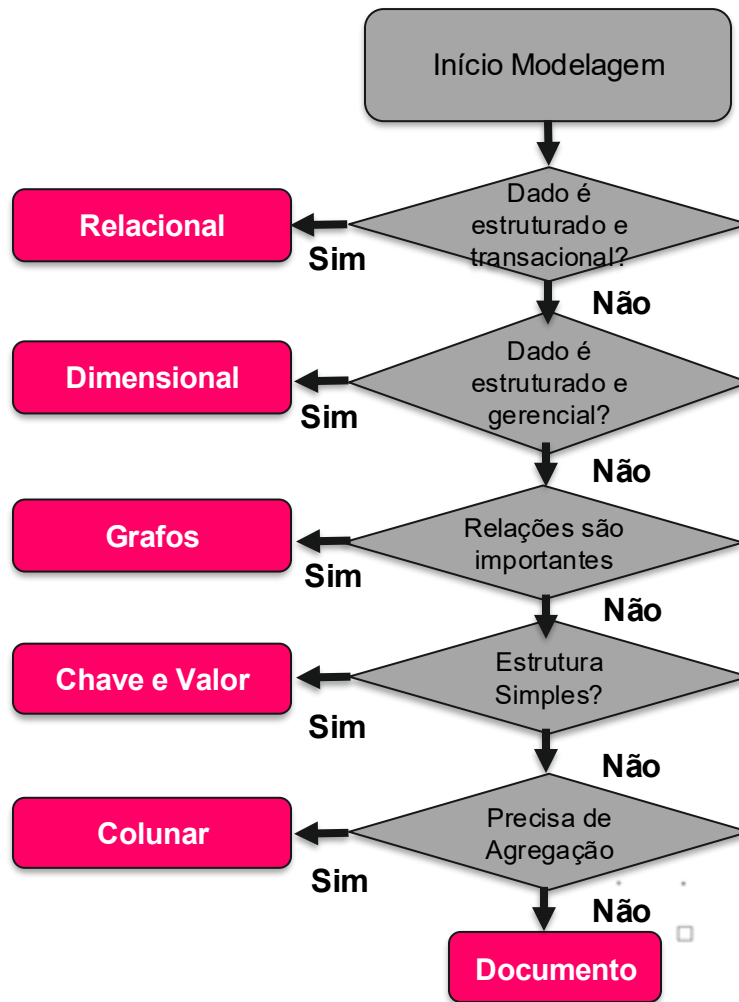


Ao criar um modelo para dados estruturados (SQL) você entende e define o que o dado significa e como os dados se relacionam entre si antes de criá-lo na base de dados. A **estrutura** do dado é fixa, **não deve mudar**.

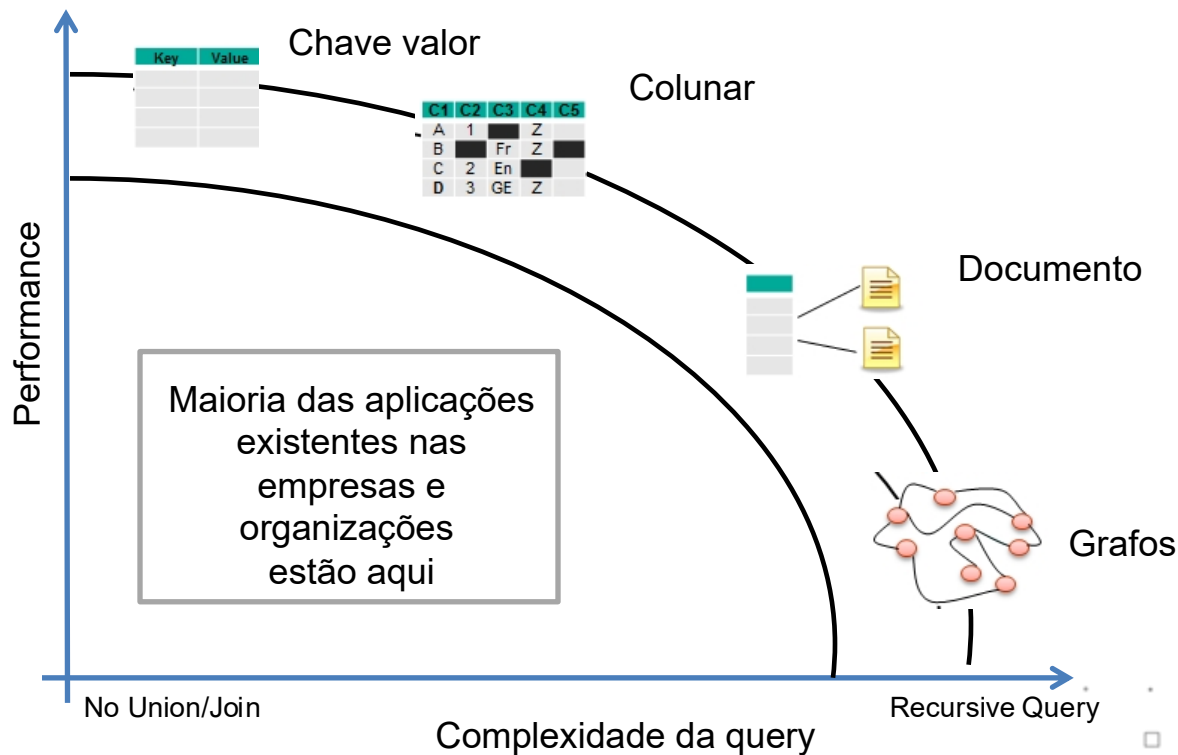
Já para dados não estruturados (NoSQL), esse entendimento prévio do dado deve ser feito, mas o modelo pode ser **alterado no momento do uso do dado**, ou seja, o modelo é “vivo” e é alterado junto com o dado.

Depende:

- Dos requisitos da aplicação;
- Do tamanho dos dados;
- Da complexidade;
- Do formato de dados;
- Da qual categoria do Teorema CAP minha aplicação se encontra



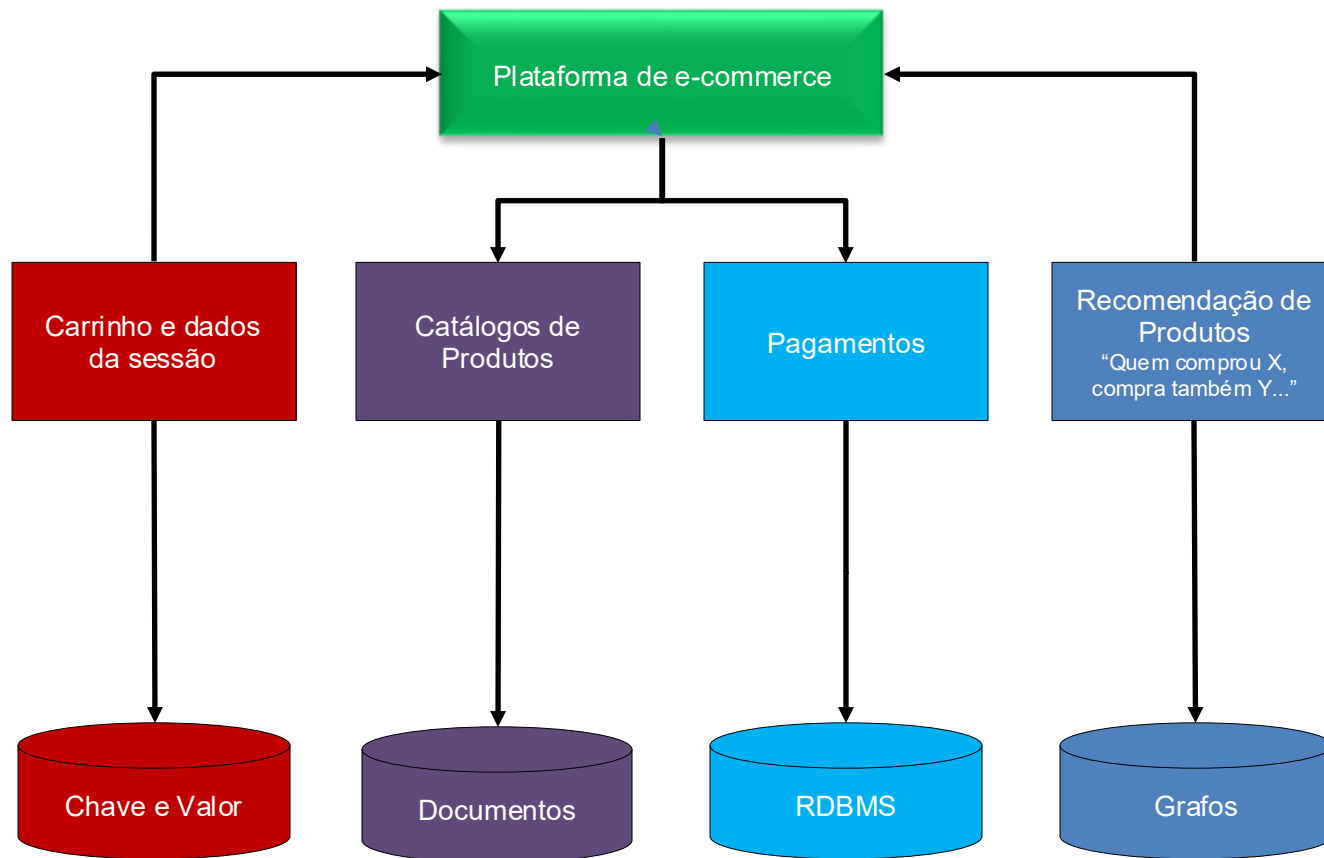
Performance VS Complexidade

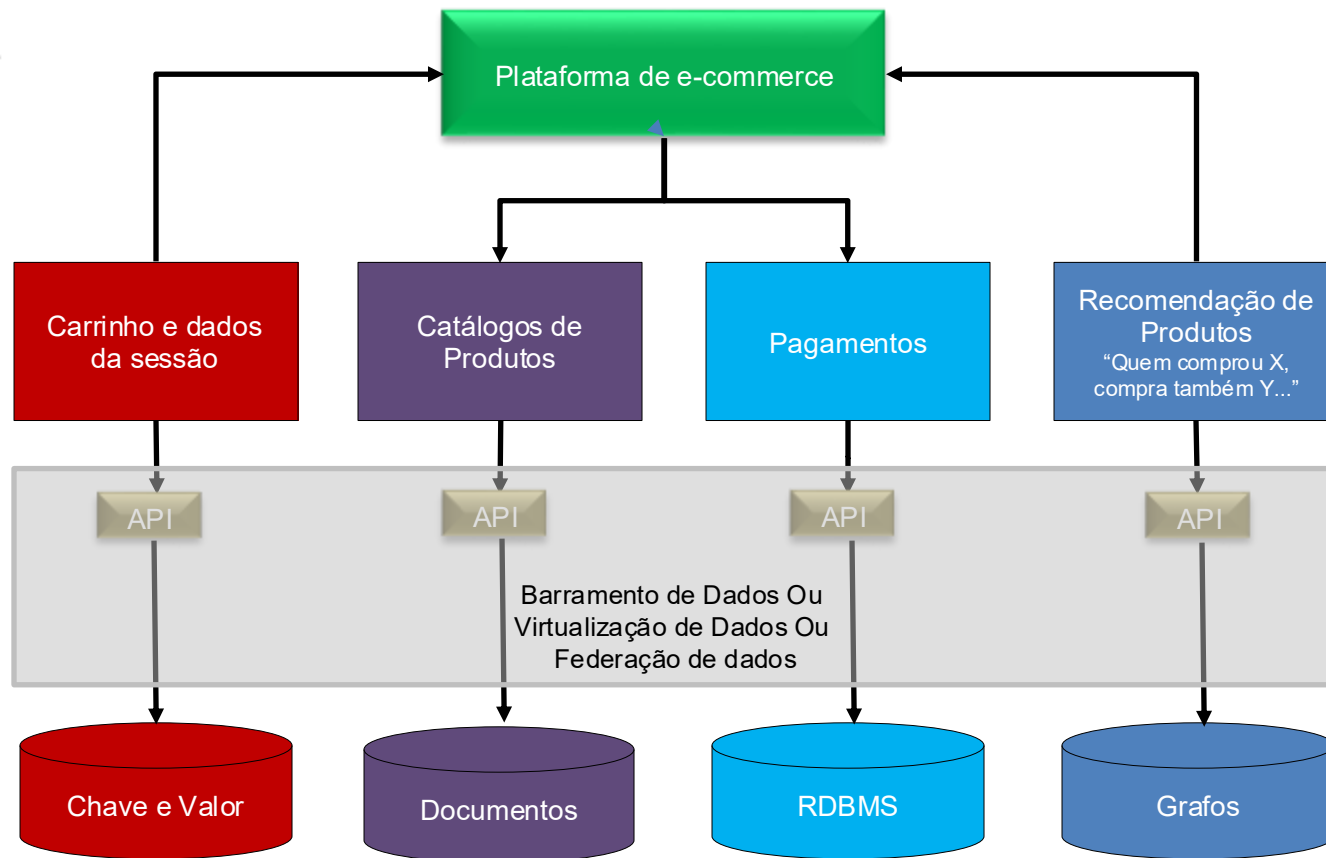


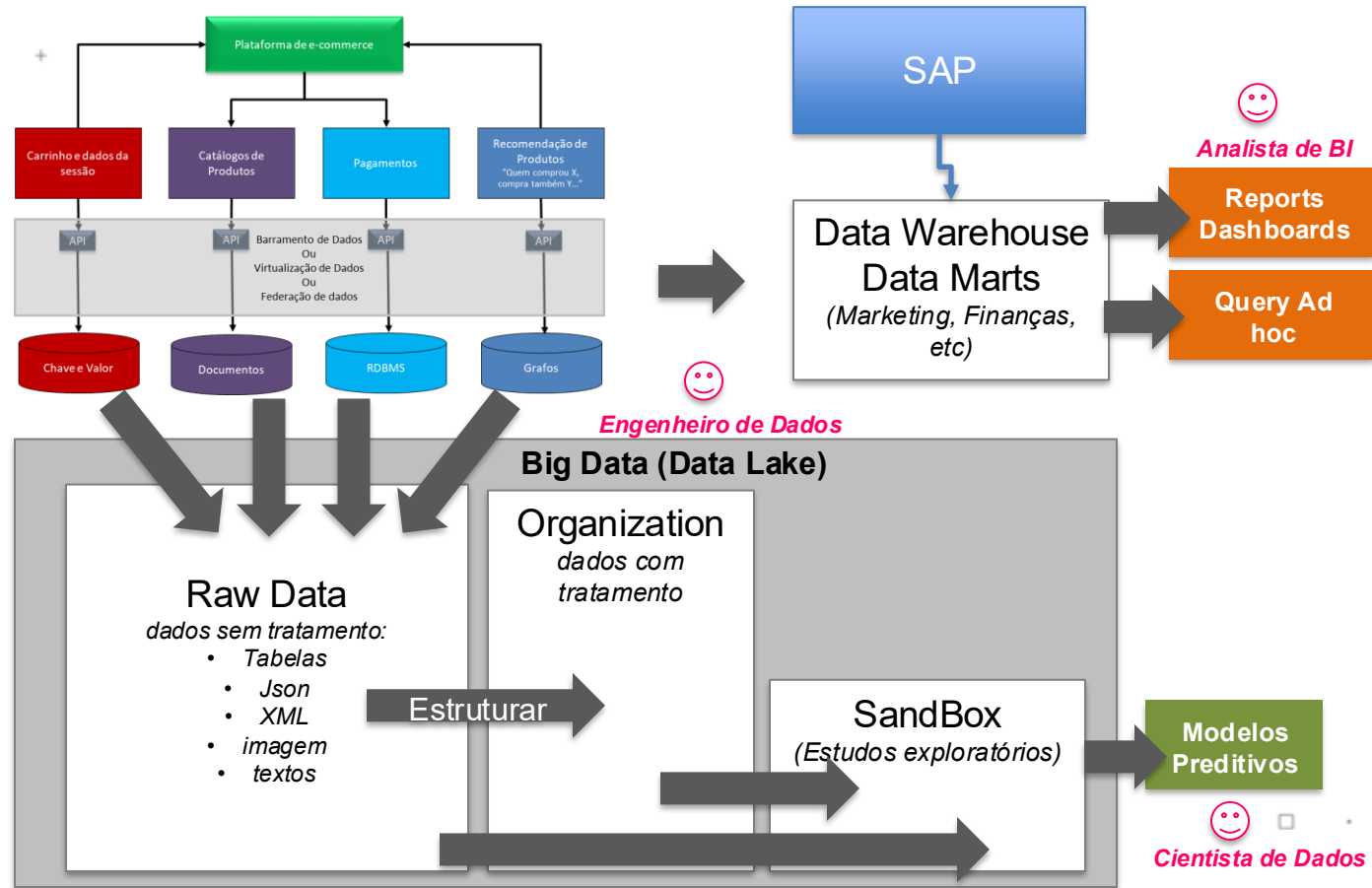
E porque se limitar
em apenas um modelo?

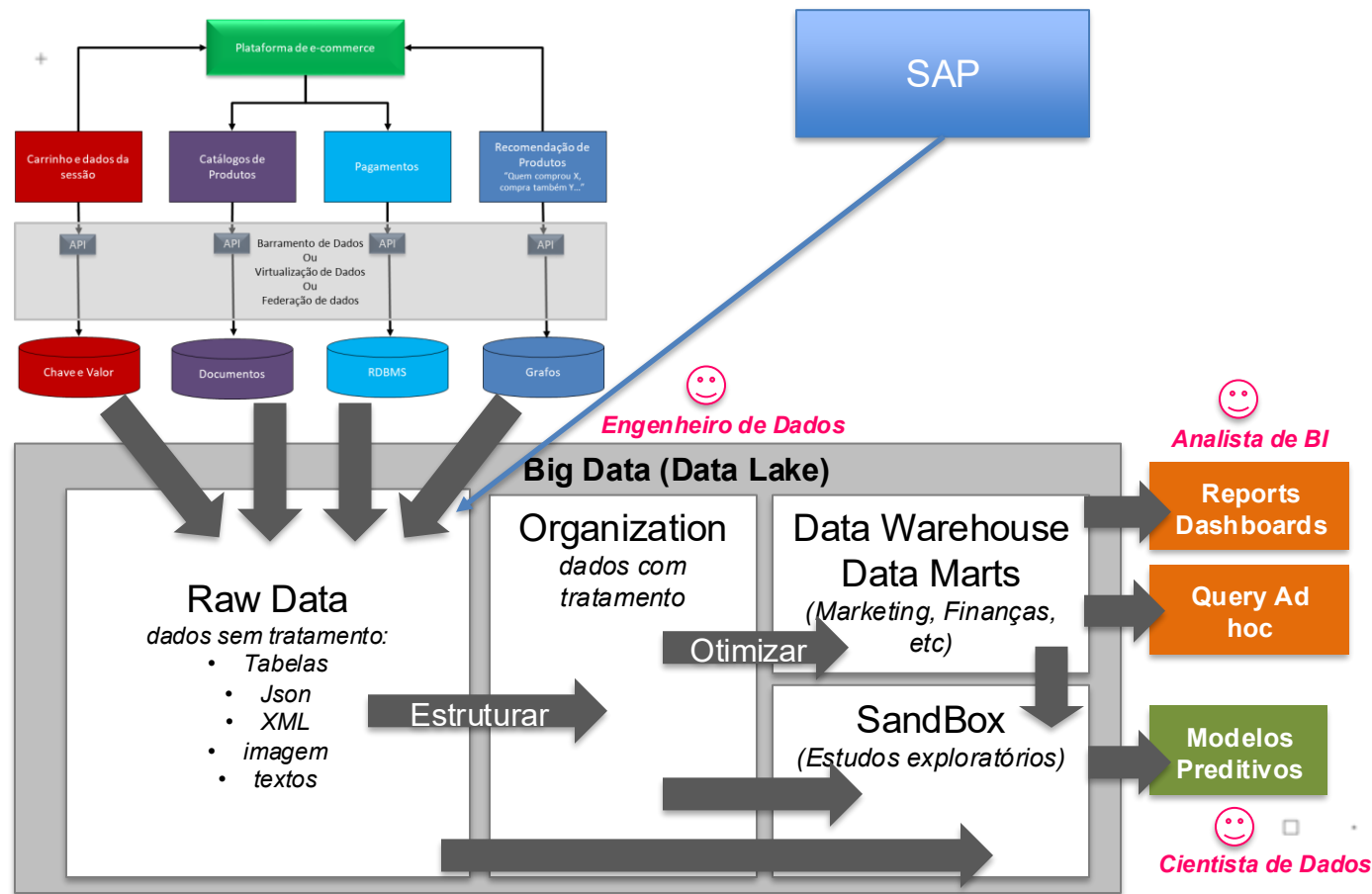
Persistência poliglota refere-se ao uso de diversos SGBD com **modelos de dados diferentes** em uma **mesma aplicação**.

Uma aplicação de persistência poliglota é projetada para se **beneficiar das características de cada SGDB** de acordo com os requisitos levantados para o projeto de construção do software.



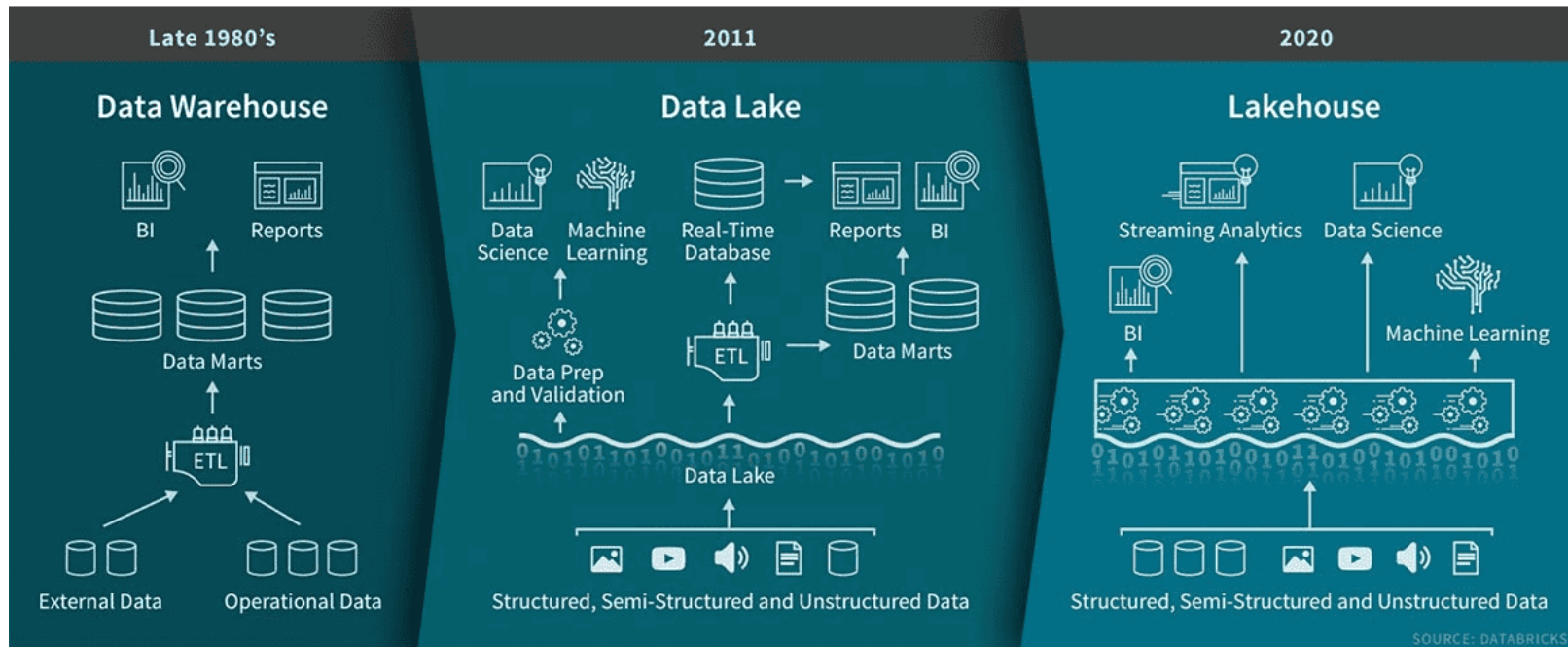




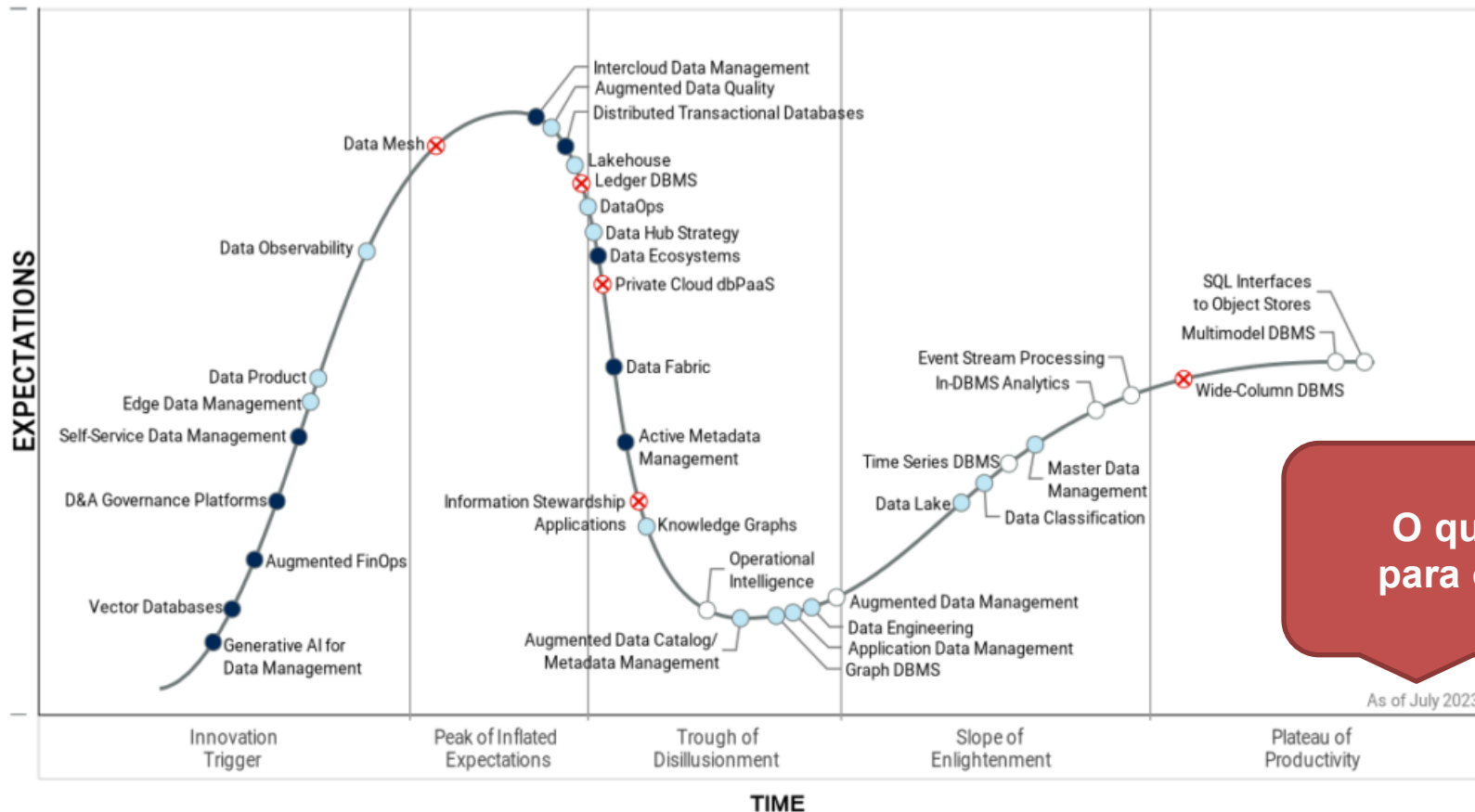


Lakehouse

Nova arquitetura aberta que combina o melhor do Data Lake com o melhor do Data Warehouse



Hype Cycle for Data Management, 2023



O que temos para o futuro?

Plateau will be reached: ○ <2 yrs. ● 2-5 yrs. ● 5-10 yrs. ▲ >10 yrs. ✗ Obsolete before plateau



- O NoSQL **não veio** para **substituir** o SQL, e sim para resolver problemas diferentes.
- **Entender o dado e conhecer o negócio** ainda é mais **importante** para desenvolver um **bom modelo**, independente da **técnica de modelagem utilizada**.

“O melhor modelo de dados é aquele que resolve o problema!”

REVISÃO



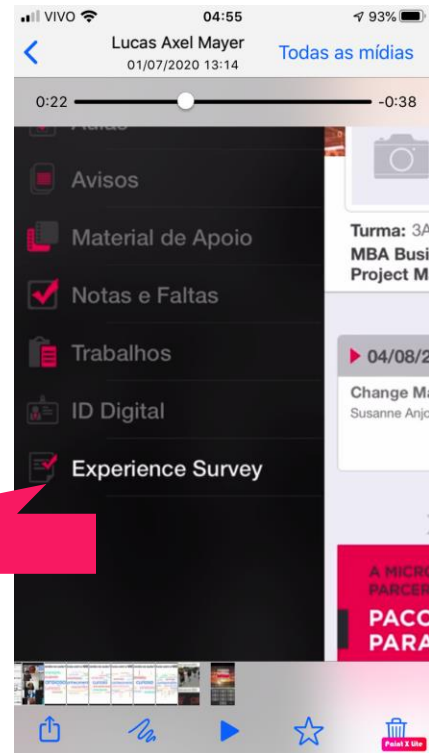
*Se liga aí,
que é hora da revisão.*

Modelagem Não Relacional

1. Definição de ACID;
2. Diferenças entre BD Centralizados e Distribuídos;
3. Teorema CAP;
4. Dados estruturados, não estruturados e semi estruturados;
5. Definição de NoSQL;
6. Tipos de Banco de Dados NoSQL:
 - Chave-valor;
 - Colunar;
 - Documentos;
 - Grafos.
7. Discussão: Qual o melhor modelo?

E não se esqueça de...

Ao final de cada aula, avaliar o professor através do aplicativo FIAPP:



Experience Survey



OBRIGADO



profeduardo.galego@fiap.com.br

FIAP MBA⁺

Copyright © 2025 | Professor Eduardo Ferreira Galego (Contribuição: ProfessoraKeylla Saes)
Todos os direitos reservados. Reprodução ou divulgação total ou parcial deste documento, é expressamente
proibido sem consentimento formal, por escrito, do professor/autor.



FIAP